

Entwicklung eines dynamischen Dungeon-Abschnitts mittels Ray Marching in Unity

Bachelorarbeit im Studiengang
Mediendesigninformatik
an der Fakultät IV –
Wirtschaft und Informatik

Hochschule Hannover
Felix Langrehr
August 2026

Designunterstützung
Mareike Werth

Inhaltsverzeichnis

Abbildungsverzeichnis	6
Glossar	8
1 Einleitung	10
1.1 Problemstellung	12
1.2 Persönliche Motivation	13
1.3 Aufbau der Arbeit	14
1.4 Einordnung der Arbeit	15
2 Ray Marching	16
2.1 Einleitung	17
2.2 Historie	18
2.2.1 Historische Entwicklung	18
2.2.2 Typische Anwendungsfelder	19
2.2.3 Bezug zu dieser Arbeit	20
2.3 Funktionsweise	22
2.4 Integration in Unity	24
2.5 Architekturüberlegungen: Objektmaterial vs. Fullscreen-Shader	25
2.6 Zwischenfazit	28
3 Grundlegende Implementierung	30
3.1 Einleitung	31
3.2 Shader-Aufbau	32
3.3 SDF-Bereich	34
3.4 Dichtewerte	36
3.5 Erstellung von 3D-Texturen	36
3.6 Zwischenfazit	38
4 Visuelle Darstellung	40
4.1 Einleitung	41
4.2 Vermeidung von Artefakten	42
4.3 Tiefenprüfung	44
4.4 Beleuchtung	46
4.4.1 Homogene Beleuchtung	46
4.4.2 Directional Light	46
4.4.3 Point Light	49
4.5 Texturierung	52
4.5.1 Triplanar Projection	52
4.5.2 Normalenapproximation	54
4.5.3 Ausnahmenbehandlung	56

4.5.4 Farbtexur	56
4.5.5 Ambient Occlusion Texur	57
4.6 Erweiterte Schattierung	58
4.7 Zwischenfazit	59
5 Physik	60
5.1 Einleitung	61
5.2 GPU-basierte Collider-Generierung	62
5.3 Charaktersteuerung	64
5.3.1 Character Controller	64
5.3.2 Bewegungsfeedback	64
5.4 Zwischenfazit	65
6 Dynamische Veränderung	66
6.1 Einleitung	67
6.2 GPU-basierter Abbau und Regeneration	68
6.3 Materialdifferenzierung	68
6.4 Zwischenfazit	70
7 Gameplay	72
7.1 Einleitung	73
7.2 Konzept	74
7.3 Ausrüstungsgegenstände	76
7.3.1 Die Lampe	76
7.3.2 Granaten	78
7.3.3 Schatzweiser	80
7.3.4 Dichtescanner	82
7.4 Zwischenfazit	84
8 Ergebnisse	86
8.1 Auswertung	88
8.2 Grenzen des Ansatzes	89
8.3 Zusammenfassung	91
8.4 Ausblick	92
8.4.1 Weiterentwicklung zum vollwertigen Spiel	92
8.4.2 Technische Tools	93
Literaturverzeichnis	94
Übersicht verwendeter Hilfsmittel	96
Übersicht verwendeter Programme	96
Bildergalerie	98

Abbildungsverzeichnis

Abbildung 1:
Funktionsweise von distanzbasiertem Ray Marching anhand eines einzelnen Kamerapixels
für eine Kugel (Quelle: Eigene Darstellung)23

Abbildung 2:
Objektmaterial von außen betrachtet (Quelle: Eigene Darstellung)26

Abbildung 3:
Objektmaterial mit Kamera an der Kante des Würfels(Quelle: Eigene Darstellung)27

Abbildung 4:
Ray Marching mit Box-SDF zum Finden des Dungeon-Bereichs, feste Schrittweiten im
Dungeon-Bereich (Quelle: Eigene Darstellung)35

Abbildung
5: In Blender modelliertes Test-Objekt (Quelle: Eigene Darstellung)37

Abbildung 6:
Aneinandergereihte Volumen-Slices (Quelle: Eigene Darstellung)37

Abbildung 7:
Sichtbare Streifen-Artefakte (Quelle: Eigene Darstellung)42

Abbildung 8:
Abstufungs-Optimierung (Quelle: Eigene Darstellung)43

Abbildung 9:
Ein sich teilweise im Ray Marching befindender Würfel ohne Tiefenprüfung
(Quelle: Eigene Darstellung)45

Abbildung 10:
Ein sich teilweise im Ray Marching befindender Würfel mit Tiefenprüfung
(Quelle: Eigene Darstellung)45

Abbildung 11:
Beleuchtung des Dungeons durch Directional Light (Quelle: Eigene Darstellung)47

Abbildung 12:
Schatten von Polygon-Objekt auf dem Dungeon, Schatten vom Dungeon auf dem Polygon-
Objekt (Quelle: Eigene Darstellung)48

Abbildung 13:
Point Light Einfluss auf die Beleuchtung des Dungeons (Quelle: Eigene Darstellung)50

Abbildung 14:
Schematische Darstellung der Normalenapproximation (Quelle: Eigene Darstellung)55

Abbildung 15:
Texturierung mittels triplanar projection (Quelle: Eigene Darstellung)56

Abbildung 16:
Einfluss der Ambient Occlusion Map (Quelle: Eigene Darstellung)57

Abbildung 17:
Verdunkelung von Stellen mit dicht beieinanderliegenden Voxeln (Ambient Occlusion)
(Quelle: Eigene Darstellung)58

Abbildung 18:
Eine Kugel mit Rigidbody und Skript, das Collider vom Dungeon um das Objekt herum
generiert (Quelle: Eigene Darstellung)63

Abbildung 19:
Teilweise abgebauter Dungeon mit verschiedenen Materialien
(Quelle: Eigene Darstellung)69

Abbildung 20:
Lampe -Texturiertes 3D-Modell (Quelle: Eigene Darstellung)76

Abbildung 21:
Lampe - Zur Laufzeit (Quelle: Eigene Darstellung)77

Abbildung 22:
Granate - Texturiertes 3D-Modell (Quelle: Eigene Darstellung)78

Abbildung 23:
Granate - Zur Laufzeit (Quelle: Eigene Darstellung)79

Abbildung 24:
Schatzweiser - Texturiertes 3D-Modell (Quelle: Eigene Darstellung)80

Abbildung 25:
Schatzweiser - Zur Laufzeit (Quelle: Eigene Darstellung)81

Abbildung 26:
Dichtescanner - Texturiertes 3D-Modell (Quelle: Eigene Darstellung)82

Abbildung 27:
Dichtescanner - Zur Laufzeit (Quelle: Eigene Darstellung)83

Glossar

Ambient Occlusion

Renderingtechnik zur Verdunklung von Ecken, Kanten und Schnittpunkten

Compute Shader

Ein Programm, das auf der Grafikkarte läuft, unabhängig von der Darstellung konkreter Geometrie

Depth Buffer

Ein Set an Bilddaten, bei denen jeder Pixel statt der Farbe die Distanz der gerenderten Stelle nicht-linear als Grauwert angibt

Engine

Programm, das primär der Entwicklung von Videospielen dient

Heightmap

Eine Graustufen-Textur, deren Werte die Position von Vertices eines Objektes verschieben

Pixel

Ein einzelner Punkt eines Bildes mit einem Farbwert

Polygon

Eine Fläche aus Punkten im dreidimensionalen Raum, mehrere davon können ein 3D-Objekt darstellen

Rasterisierung

Umwandlung von Linien und Dreiecken von 3D-Modellen in ein festes Pixelraster

Rendering

auch Bildsynthese; Die Erzeugung eines Bildes aus Rohdaten wie 3D-Modellen, Texturen und Lichtquellen

Shader

Kleines Programm, das auf der Grafikkarte läuft, häufig um das Aussehen von Pixeln auf dem Bildschirm zu berechnen

Signed Distance Field

Textur, deren Pixelwerte die Distanz zur Oberfläche anstelle von Farben enthalten

Signed Distance Function

Mathematische Funktion, mit der bestimmt werden kann, wie weit ein Punkt von der Kante des mit der Funktion beschriebenen Volumens entfernt ist

Universal Render Pipeline (URP)

Eine Alternative Rendering Pipeline, die inzwischen die Built-In Pipeline von Unity als Standard abgelöst hat

Volumetrisch

Darstellung von ganzen Volumen (Körperinhalten) anstatt nur der äußeren Oberfläche

Vertex

Einzelner Punkt im dreidimensionalen Raum, mehrere (Vertices) können ein Polygon bilden

Voxel

Pixel im dreidimensionalen Raum (Volumen Pixel)

1 Einleitung

1.1 Problemstellung

Die Generierung und Darstellung von Spielumgebungen in Echtzeit-Engines wie Unity erfolgt aktuell überwiegend auf Basis traditioneller polygonaler Ansätze. Dabei dominieren zwei Verfahren: Terrain-Systeme und vorgefertigte 3D-Modelle. Terrain-Systeme modellieren Umgebungen primär als zweidimensionale Heightmaps, die die Vertexpositionen entlang der vertikalen Achse verschieben (vgl. Unity Technologies, 2026f). Dieser Ansatz ist für offene, horizontale Landschaften effizient, stößt jedoch bei geschlossenen, mehrschichtigen oder vollständig umschließenden Strukturen an fundamentale Grenzen. Der Einsatz statischer 3D-Modelle oder 3D Modelle als modulare Bauteile ermöglicht zwar die Konstruktion komplexerer Innenräume, erfordert jedoch einen hohen Vorproduktionsaufwand und bietet nur begrenzte Laufzeitflexibilität. Insbesondere dynamische Veränderungen der Umgebung wie das gezielte Abbauen von Wänden oder das organische Nachwachsen von Strukturen, lassen sich mit polygonalen Geometrien nur unter erheblichen Performance-Einbußen oder durch aufwendige, laufzeitkritische Mesh-Manipulationen umsetzen.

Als Alternative zu expliziter Polygon-Geometrie etabliert sich Ray Marching als leistungsfähiges Verfahren für volumetrisches Rendering. Technisch und praktisch wird das Verfahren vorrangig für atmosphärische Effekte wie Wolken, Nebel, Rauch oder volumetrische Beleuchtung eingesetzt. Diese Anwendungen sind alle rein visuell

und erfordern keine physikalische Interaktion, Kollisionsabfrage oder Modifikationen durch den Spieler. Die Übertragung von Ray Marching auf vollständige, interaktive Spielumgebungen bleibt daher ein eher unerschlossenes Anwendungsfeld. Die Problemstellung dieser Arbeit liegt darin, die technischen und konzeptionellen Limitationen dieser Übertragung zu lösen. Dabei geht es vor allem um folgende Aspekte: die performante GPU-basierte Verarbeitung von volumetrischen Daten, die Integration in die Rendering Pipeline von Unity, die Abbildung physikalischer Kollisionen von der Spielumgebung mit Unity-eigenen Physik sowie die laufzeitfähige, dynamische Modifikation des Volumens.

Ziel dieser Arbeit ist es, einen technischen und konzeptionellen Ansatz zu entwickeln, der Ray Marching als Grundlage für einen veränderlichen Dungeon-Abschnitt nutzbar macht und in einer kleinen technischen Demo zeigt. Es wird untersucht, wie sich Daten für die Umgebung für das Ray Marching nutzbar machen lassen, dass sie visuell kohärent und überzeugend und interaktiv sowie veränderbar sind. Die Arbeit liefert eine praxisorientierte Methodik zur Realisierung dynamischer Umgebungen jenseits traditioneller Mesh-basierter Ansätze und dokumentiert die notwendigen Architektur-, Shader- und Physik-Integrationen für eine funktionale Demo.

1.2 Persönliche Motivation

Doch wieso habe ich mir ausgerechnet Ray Marching als zentralen Bestandteil meiner Arbeit gesetzt? Meine Berührungspunkte mit der Spieleentwicklung starten vergleichsweise früh. Schon als ich 13 Jahre alt war, wollte ich die Spiele nicht nur spielen, sondern auch verstehen, wie es überhaupt möglich ist, solche Welten zu erschaffen. Damals wurde das Problem der Wolkendarstellung in Spielen meistens durch eine Skybox, also eine Cubemap, die den Spieler umgibt, gelöst. Manches mal beinhaltete die Skybox aber auch nur den wolkenlosen Himmel und die Wolken waren eine zusätzliche, sich zeitlich durch Rauschtexturen ändernde Fläche, die weit über dem Spieler platziert war. Gedanken habe ich mir über volumetrische Effekte damals nicht gemacht, denn das nicht-Vorhandensein war der Zustand, den ich gewohnt war. Dass es auch anders geht, zeigte mir eines der Spiele, die ich am meisten in meinem Leben gespielt habe: Ark: Survival Evolved. In diesem Spiel kann man man die Welt erkunden, Ressourcen sammeln, Strukturen bauen und Dinos zähmen und auf ihnen durch die Welt reiten. Ich war sehr überrascht, als ich das erste Mal einen Flugsaurier zähmte und in den Himmel, durch die Wolken fliegen konnte. Mit den mir bis dato bekannten Möglichkeiten, hatte ich keine Ahnung, wie die Entwickler das bewerkstelligt haben.

Auf meiner Suche nach Antworten stieß ich irgendwann auf ein Video von Sebastian Lague mit dem Titel „Coding Adventure: Clouds“ (Lague, 2019), in dem er Ray Marching anwendet, um realistische Wolken darzustellen. Das war mehrere Jahre vor Beginn meines Studiums und mir fehlte Wissen, um in der Lage zu sein, dem Video zu folgen. Auch wenn das Video sehr interessant anzuschauen ist, so ist es nicht zum Nachmachen geeignet, wenn man sich mit der Materie nicht auskennt. Nun bin ich am Ende meines Medieninformatik Studiums angelangt und traute mir zu, meine eigene Umsetzung dieser Technik anzugehen. Da sie für Wolken inzwischen bereits standardmäßig in Game Engines integriert ist, ist mein Ziel die Erforschung einer anderen Anwendungsmöglichkeit, die mir als eine logische Möglichkeit zur Verwendung von Ray Marching erschien.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in einen theoretischen, einen implementierungsorientierten und einen evaluierenden Teil.

Kapitel 2 legt das technische Fundament, indem es die historische Entwicklung und typischen Anwendungsfelder von Ray Marching erläutert und anschließend die Funktionsweise beschreibt. Nach der Integration in die Unity Engine folgt ein sehr zentraler Abschnitt, die architektonische Überlegung, einen Fullscreen Shader anstelle eines herkömmlichen Shaders für ein Objektmaterial zu verwenden.

Kapitel 3 widmet sich der grundlegenden Implementierung des Ray Marching Verfahrens. Hier wird der Aufbau des Shaders sowie die Einbindung des Shadercodes in einen Shader Graph erläutert. Zudem werden die effiziente Begrenzung des Berechnungsbereichs mittels Box-SDF, die Akkumulation von Dichtewerten und der Workflow zur Generierung der zugrundeliegenden 3D-Texturen vorgestellt.

Aufbauend darauf fokussiert sich Kapitel 4 auf die visuelle Darstellung und die Integration in die Rendering-Pipeline. Es werden Techniken zur Vermeidung von Schrittweitenartefakten, die korrekte Tiefenprüfung mit polygonalen Objekten sowie die Implementierung von Beleuchtungssystemen (Directional und Point Light) behandelt. Darüber hinaus werden die Texturierung mittels Triplanar Projection, die Approximation von Normalenvek-

toren und der Einsatz von Ambient Occlusion zur Steigerung der räumlichen Tiefenwahrnehmung erläutert.

Kapitel 5 adressiert die physikalische Interaktion mit dem volumetrischen Raum. Da Ray Marching keine explizite Mesh-Geometrie liefert, wird eine GPU-basierte Collider-Generierung mittels Compute Shader vorgestellt, die eine Kopplung mit dem Unity-Physiksystem ermöglicht. Ergänzend wird der Rigidbody-basierte Charaktercontroller sowie das Bewegungsfeedback zur Steigerung der Spielimmersion beschrieben.

In Kapitel 6 wird die laufzeitfähige Dynamik des Dungeons behandelt. Es wird dargelegt, wie der GPU-basierte Abbau und die kontinuierliche Regeneration der 3D-Textur implementiert sind. Zudem wird die kanalabhängige Materialdifferenzierung beschrieben, die unterschiedliche Widerstandswerte und Regenerationsgeschwindigkeiten für verschiedene Dungeon-Bereiche ermöglicht. Kapitel 7 überträgt die technischen Grundlagen in ein konkretes Gameplay-Konzept. Das explorative Spielprinzip wird vorgestellt und die Implementierung der zentralen Austrüstungsgegenstände beschrieben. Diese Instrumente verbinden technische Funktionalität mit spielmechanischer Navigation und Interaktion.

Die Arbeit schließt mit Kapitel 8 ab, das eine Auswertung der Ergebnisse liefert. Die visuellen und technischen Umsetzungserfolge werden analysiert, die Grenzen des Ansatzes kritisch reflektiert und eine zusammenfassende Bewertung getroffen. Ein abschließender Ausblick gibt Perspektiven für die Weiterentwicklung zum vollwertigen Spiel sowie für die Nutzung der entwickelten Shadertechniken als eigenständige Entwicklungstools.

1.4 Einordnung der Arbeit

Diese Arbeit positioniert sich an der Schnittstelle zwischen computergestützter Grafikforschung und praktischer Spieleentwicklung. Während Ray Marching in der Fachliteratur und Praxis hauptsächlich für volumetrische Effekte wie Nebel, Rauch oder Wolken etabliert ist, untersucht diese Arbeit die Eignung des Verfahrens als Methode zur Darstellung interaktiver, räumlich umschließender Spielumgebungen. Damit wird eine bestehende Lücke adressiert: Herkömmliche Ansätze in Echtzeit-Engines wie Unity basieren auf statischen Meshes oder Heightmap-Terrains (vgl. Unity Technologies, 2026f), die bei dynamisch veränderlichen, geschlossenen Räumen an Flexibilitätsgrenzen stoßen. Diese Arbeit demonstriert, dass Ray Marching in Kombination mit modernen GPU-Features wie Compute Shadern und Funktionen der Unity Universal Rendering Pipeline ein leistungsfähiges Fundament für volumetrische

Geometrien bieten kann, die sich in Echtzeit entfernen und regenerieren sowie mit Physik interagieren lassen.

Wissenschaftlich trägt die Arbeit zur Praxisorientierung fortgeschrittener Rendering-Techniken bei, indem sie theoretische Konzepte der Signed Distance Functions und volumetrischen Strahlenverfolgung in einen konkreten Entwicklungsworkflow überführt. Technisch wird gezeigt, wie Shader-Graph basierte Architekturen mit Custom Function Nodes erweitert werden können, um komplexe Berechnungen performant auf der GPU auszuführen. Die Implementierung einer GPU-gesteuerten Collider-Generierung und die kanalabhängige Materialsteuerung erweitern den Anwendungshorizont von Ray Marching über reine Visualisierung hinaus zu einem interaktiven, gameplayrelevanten System.

Im Kontext der Mediendesigninformatik verbindet die Arbeit damit grafische Algorithmen, technische Implementierung und konzeptionelle Spieleentwicklung. Sie liefert einen Referenzansatz für Entwickler und andere Forschende, die dynamische Umgebungen jenseits traditioneller Polygonmodelle realisieren möchten. Sie zeigt aber auch die Grenzen, mit denen man mit Anwendung der Technik umzugehen lernen muss.

2 Ray Marching

2.1 Einleitung

Bevor die technische Umsetzung des dynamischen Dungeons mittels Ray Marching detailliert beschrieben wird, ist ein grundlegendes Verständnis der zugrundeliegenden Rendering-Technik notwendig. Ray Marching stellt ein alternatives Verfahren zur klassischen polygonbasierten Geometriedarstellung in der Computergrafik dar, das insbesondere für volumetrische und geschlossene Strukturen geeignet ist.

Dieses Kapitel legt das theoretische Fundament, indem es die historische Entwicklung und die typischen Anwendungsfelder des Verfahrens erläutert. Anschließend wird die Funktionsweise sowie die Integration in die Umgebung der Unity Engine ausgeführt. Zum Schluss dieses Kapitels werden die architektonischen Überlegungen beleuchtet, die zur Entscheidung für einen Fullscreen-Shader führten.

2.2 Historie

2.2.1 Historische Entwicklung

Die theoretischen Grundlagen des Ray Marching lassen sich auf frühe Ansätze zur impliziten Geometriedarstellung zurückführen. In der Publikation „Field functions for implicit surfaces“ von Brian Wyvill und Georg Wyvill (vgl. Brian Wyvill, 1989, pp. 75 - 81) werden mathematische Funktionen zur Modellierung weicher Übergänge zwischen inneren und äußeren Regionen eines Volumens eingesetzt. Ihre Arbeit basiert auf vorangegangenen Darstellungen von Blinn, der 1982 versuchte, energetische Oberflächen in Molekülen darzustellen, sowie Middleditch und Sears, die 1985 eine Technik für Darstellungen von sanften Übergängen in solchen Systemen entwickelten.

Brian und Georg Wyvill selbst war die bis dahin übliche Beschreibung von dreidimensionalen Objekten mit Polygonen oder Splines zu wenig. Mit ihrer impliziten Darstellung von Geometrien wollten sie vor allem Objekte wie Flüssigkeiten darstellen können, die sich mit der Zeit verändern. Obwohl der Begriff „Ray Marching“ an dieser Stelle nicht verwendet wird, bildet die beschriebene kontinuierliche Feldrepräsentation die Basis für moderne Signed Distance Functions (SDFs), die heute oft zur effizienten Strahlverfolgung genutzt werden (vgl. Brian Wyvill, 1989, pp. 75 - 81).

Die von Brian und Georg Wyvill ausgearbeitete Variante zur impliziten Beschreibung von Oberflächen war aber keine Renderinganweisung,

sondern die geometrische Repräsentation. Die algorithmische Weiterentwicklung erfolgt 1995 durch John C. Hart mit der Einführung des „Sphere Tracing“ (vgl. Hart, 1995). Dieses Verfahren formalisiert das Prinzip der iterativen Strahlenverfolgung durch SDFs und ermöglicht das Rendern von impliziten Oberflächen ohne Polygonisierung. Vor der Weiterentwicklung von Hart wurden implizite Oberflächenbeschreibung noch in Polygone umgewandelt, um sie rendern zu können. Hart war damit aber nicht zufrieden, die für die Polygonisierung benötigte Zeit und ihre Ungenauigkeit hat er durch die Entwicklung des Sphere-Tracings behoben (vgl. Hart, 1995).

Valve, das Unternehmen hinter der weltbekannten Spiele-Vertriebsplattform Steam sowie erfolgreichen Spielen wie Portal, Half-Life und Counter Strike, suchte im Jahr 2007 nach einer Möglichkeit, Vektorgrafiken wie Texte oder Logos in 3D Umgebungen mit ihrer eigenen Source Engine besser darzustellen. Aufgrund der Hardware-Limitationen konnte man damals keine hochauflösenden Texturen für solche Zwecke verwenden. Chris Green, Mitarbeiter bei Valve, überlegte sich stattdessen die hochauflösenden Grafiken in Signed Distance Fields umzurechnen und statt der eigentlichen Textur diese zu rendern. So konnte er mit einer sehr niedrigen Auflösung, die normalerweise einen pixelig-verwaschenen Look produzieren würde, gestochen scharfe Kanten darstellen (vgl. Chris Green, 2007, pp. 9-16).

2.2.2 Typische Anwendungsfelder

Die von Valve erprobte Variante zur Darstellung von Texten und Icons wird auch heute noch aktiv verwendet. TextMeshPro, der inzwischen in Unity integrierte Text-Renderer ist mittels SDF Fonts in der Lage, bei beliebigen Vergrößerungen Texte gestochen scharf darzustellen (Unity Technologies, 2023).

Doch vor allem für Volume Rendering ist Ray Marching bekannt. Auf der Game Developers Conference 2005 hat NVIDIA Ray Marching als Alternative zu bis dahin üblicherweise verwendeten Partikelsysteme und Slices zur Volumendarstellung vorgestellt. Erst ab diesem Zeitpunkt wurde Ray Marching zur Anwendung in Echtzeitanwendungen wie Spielen praktisch brauchbar, da NVIDIA die Möglichkeit, in Shadern mit Schleifen zu programmieren, einführte (Simon Green, 2005). Seitdem wurde Ray Marching als Technik beliebter und fand für die Wolkendarstellungen in modernen Game Engines wie der Unreal Engine (Epic Games, 2026) und Unity (Unity Technologies, 2023b) Einzug.

Durchaus interessant ist auch eine Anwendung ganz abseits von spielerischem Kontext. Um Ärzten einen besseren Blick in den Körper von Patienten zu ermöglichen, kann aus Bildern, die mittels Computer Tomografie (CT) entstanden sind, ein Volumen berechnet werden, das sich durch Ray Marching darstellen lässt (vgl. S. M. Keerthana, 2025, pp. 1 - 5).

Eine maßgebliche Inspiration zur Umsetzung dieser Arbeit lieferte das Spiel „The Gunk“. In diesem spielt die Verwendung von Ray Marching eine entscheidende Rolle zur visuellen Umsetzung der Spielmechanik. Der Spieler säubert Areale von einer schleimigen Substanz, indem er sie mit mitgeführtem Gerät aufsaugt. Diese Substanz ähnelt einer organischen Flüssigkeit und diese Substanz wird mittels Ray Marching gerendert (vgl. Larsson, kein Datum). In der Veränderbarkeit der Umgebung hat diese Verwendung Ähnlichkeiten mit dieser Arbeit, jedoch mit dem Unterschied, dass diese vom Spieler entfernbare Masse nur punktuell in der Spielwelt vorkommt und nicht die gesamte Umgebung daraus besteht.

2.2.3 Bezug zu dieser Arbeit

Eine Umgebung als Volumen darzustellen, bedeutet dass die geometrische Beschreibung implizit stattfindet. Diese implizite Darstellung führt durch die Abstufungs-Optimierung in Kapitel 4.2 auch zu einer weichen Darstellung, wie sie historisch explizit gewünscht war. Das nicht-Vorhandensein von harten Kanten entspricht der gewünschten Ästhetik, der Dungeon soll sich lebendig anfühlen.

Polygone sind die Menschen inzwischen gewohnt. Wenn dem Spieler die Möglichkeit gegeben wird, selbst die Umgebung umzuformen, was bei vielen Spielen gar nicht gegeben ist, handelt es sich oft immer noch um polygonale Geometrie, die verformt wird.

Manches Mal werden volumetrische Felder verwendet wie in dieser Arbeit, wobei Ray Marching als Rendering-Technik dann meist nicht zum Einsatz kommt. Stattdessen werden aus den volumetrischen Feldern wieder Polygone erzeugt. Als Beispielanwender solcher Verfahren sei an dieser Stelle das Spiel „No Man’s Sky“ erwähnt, das einen dual marching cubes voxel meshing Algorithmus verwendet, der bei Veränderungen des Terrains neue Polygon-Meshes aus einem Voxel-raster erzeugt (vgl. Hello Games, 2024).

Diese Umwandlung hat aus Gründen der effizienten Darstellung von Polygonen durch die Grafikkarte ihre Daseinsberechtigung. Jedoch erkennt der Spieler die Kanten und wird so daran erinnert, dass er sich in einer Welt aus Dreiecken befindet. Indem mit dieser Arbeit ein unkonventioneller Ansatz gewählt wird, bei der Darstellung der Spielwelt auf Dreiecke zu verzichten, erlebt der Spieler eine neue Art der Immersion.

Diese Arbeit hat aber nicht den Anspruch, eine allgemein „bessere“ Form der Umgebungsdarstellung zu sein und zukünftig in jedem Spiel Anwendung zu finden. Es geht vielmehr darum, dem Spieler mal etwas anderes zu zeigen, was er noch nicht aus jedem Spiel kennt und deshalb nicht automatisch gedanklich als Spielegrafik einordnet. Es lädt den Spieler ein, sowohl die Spielwelt als auch das System dahinter zu erkunden.

Diese Arbeit entsteht auch mit der Hoffnung, dass technisch Interessierte, die sich nur die Demo anschauen, aber die Arbeit noch nicht gelesen haben, hinterfragen, wie diese Darstellung möglich ist und so selbst Interesse entwickeln, sich näher damit zu beschäftigen.

Wird bedacht, dass echte Materialien aus unzähligen Atomen bestehen und echte Objekte somit im Inneren nicht hohl sind, wie es bei Darstellungen durch Polygone der Fall ist, so ist die Annahme, dass die Darstellung als Volumen der eigentlich richtigen Darstellung entspricht, gar nicht so abwegig. Bei Objekten, die sich nie verändern, ist das egal. Da ergibt es Sinn, nur die äußeren Flächen darzustellen und sich das Innere zu sparen.

Aus diesem Grund fiel auch die Entscheidung, Polygon-Objekte aus diesem Projekt nicht zu verbannen, sondern darauf zu achten, sie in die Umgebung integrierbar zu machen. Bei den später behandelten Ausrüstungsgegenständen handelt es sich um normale polygonale 3D Modelle mit üblichen Materialeigenschaften für Flächen. Für einzelne Objekte ist so ein höherer Detailgrad möglich und sie sind durch herkömmliche Workflows einfacher zu erstellen.

2.3 Funktionsweise

Das Ray Marching Verfahren basiert auf der iterativen Verschiebung eines Punktes entlang eines Strahls von der Kamera aus durch den dreidimensionalen Raum. Für jeden Bildpunkt wird ein Strahl initialisiert und schrittweise entlang der Richtung vorangetrieben. An jeder Iterationsposition werden Berechnungen durchgeführt, die die Schrittweite für den nächsten Schritt definieren oder die akkumulierte Farbe oder Dichte des Pixels beeinflussen. Bei Verwendung von SDFs wird der Strahl maximal um den abgetasteten Abstandswert fortgeschritten. Diese Eigenschaft verhindert das Eindringen in Geometrien und minimiert die Anzahl notwendiger Rechenschritte, was zu einer signifikanten Performancesteigerung im Vergleich zu festen Schrittweiten führt (vgl. Hart, 1995, pp. 74 - 76).

Grafikkarten sind eigentlich auf Polygon-Rasterisierung ausgelegt. Sie wurden speziell dafür optimiert, Dreiecke zu rasterisieren (vgl. Samuli Laine, 2011, pp. 79 - 83). Beim Ray Marching muss unbedingt selbst auf Performance-Optimierungen geachtet werden, denn wenn die Berechnungen pro Pixel zu aufwändig werden, kann das schnell zu großen Performanceverlusten führen.

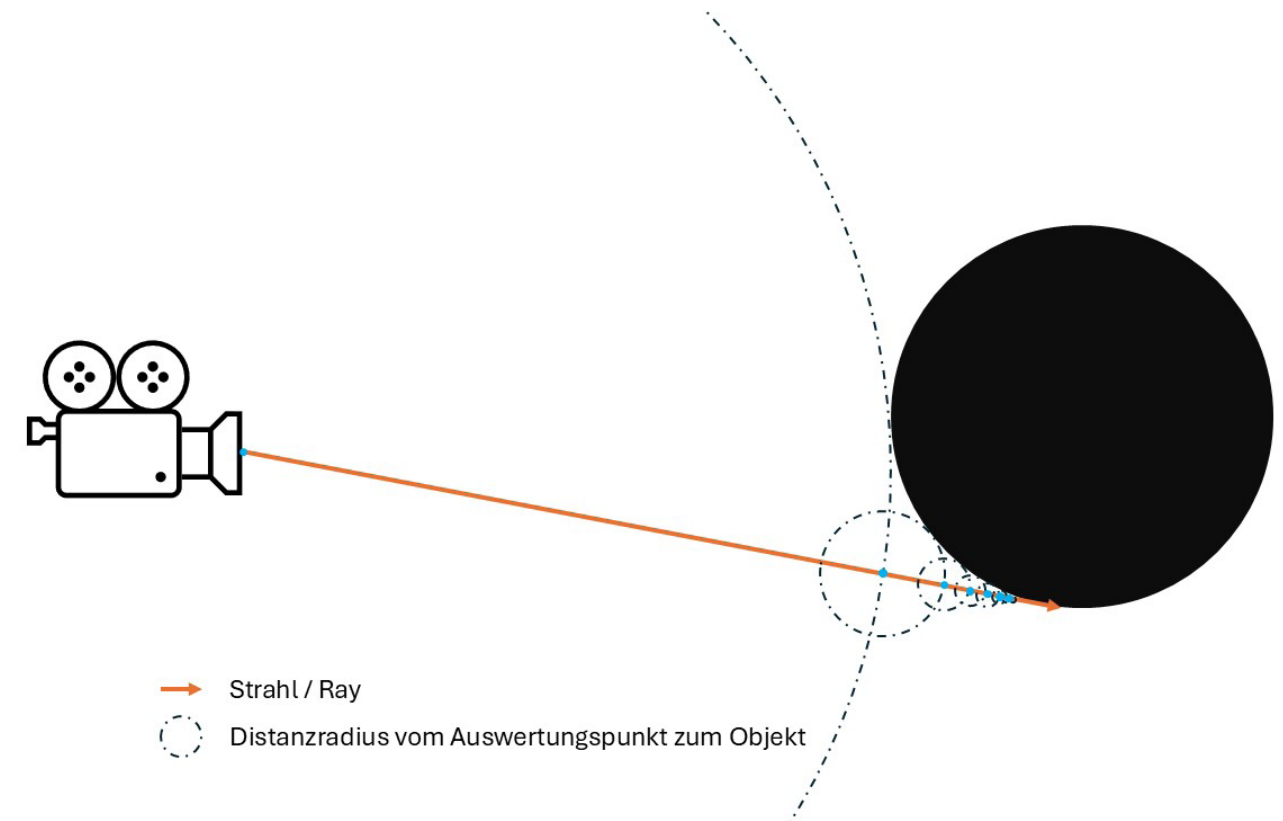


Abbildung 1: Funktionsweise von distanzbasiertem Ray Marching anhand eines einzelnen Kamerapixels für eine Kugel
(Quelle: Eigene Darstellung)

2.4 Integration in Unity

Für die Speicherung und den GPU-seitigen Zugriff auf volumetrische Daten nutzt Unity 3D-Texturen (vgl. Unity Technologies, 2026a). Die diskreten räumlichen Einheiten innerhalb dieser Textur werden als Voxel (Volume Pixel) bezeichnet. Im Gegensatz zu 2D-Texturen ermöglichen 3D-Texturen eine direkte, hardwarebeschleunigte Abfrage in alle drei Raumdimensionen, was sie zur idealen Datenstruktur für volumetrische Strahlverfolgung macht.

Während es früher noch üblich war, Shader komplett mit GLSL oder HLSL selbst zu schreiben, gibt es in Unity inzwischen Shader Graph. Dieser erlaubt das visuelle Erstellen und Bearbeiten von eigenen Shadern durch ein Node-basiertes System. Auch wenn Unity die Shader intern wieder in Shader-Code umwandelt, ist Shader Graph als modernes System kaum noch wegzudenken. Um darin Ray Marching Berechnungen durchzuführen, eignet sich dieses System aber nur

bedingt, da dieses Node System aufgrund seines visuellen Charakters Einschränkungen unterliegt. Es ist nicht möglich, Schleifen zu verwenden, die für das Ray Marching aber zwingend erforderlich sind. Es ist dennoch nicht nötig komplett auf Shader Graph zu verzichten, denn die Lösung dafür ist die Custom Function Node. In dieser kann man entweder direkt seinen eigenen HLSL Code schreiben oder eine Datei einbinden, die den Code enthält und darin definierte Funktionen verwenden. Die Entscheidung fiel hierbei auf das Einbinden einer Datei, da der Ray Marching Code erwartbar etwas länger wird und die Übersichtlichkeit ansonsten stark darunter leiden würde.

Für dieses Projekt wird Unity 6 LTS verwendet, wobei LTS für „long term support“ steht, also eine Version, die fortlaufend mit Updates versorgt und möglichst stabil gehalten wird.

2.5 Architekturüberlegungen: Objektmaterial vs. Fullscreen-Shader

Eine erste architektonische Überlegung sah vor, das Ray-Marching-Verfahren als Materialshader auf ein polygonales Box-Objekt anzuwenden. Dieser Ansatz erscheint auf den ersten Blick naheliegend, da die World-Position des Pixels im Fragment Shader bekannt ist und die Kamerarichtung eine direkte Strahlverfolgung ermöglicht.

Solange die Kamera sich außerhalb des Volumens befindet, lassen sich Berechnungen – beispielsweise das Samplen einer 3D-Textur – erwartungsgemäß durchführen und die Pixelfarbe entsprechend bestimmen. Sobald sich die Kamera jedoch in das Innere des Volumens bewegt, zeigt sich eine fundamentale Limitierung: Der Strahl wird erst an der Oberfläche des polygonalen Objekts initialisiert. Das Innere bleibt damit unzugänglich für die volumetrische Berechnung, was eine immersive Bewegung durch den Dungeon verhindert.

Um diese Einschränkung zu umgehen, wird auf einen Fullscreen-Shader umgestellt. Dieser Ansatz entkoppelt die Ray-Marching-Berechnung von polygonalen Geometrien. Jeder Bildpunkt wird unabhängig von der Szene direkt verarbeitet, wodurch das gesamte Volumen durchgängig berechnet werden kann. Der Shader wird als separater Rendering-Pass vor den Post-Processing-Effekten eingefügt. Dadurch unterliegt das Ergebnis der volumetrischen Berechnung denselben visuellen Nachbearbeitungen wie der Rest der Szene, was eine konsistente Darstellung sicherstellt.

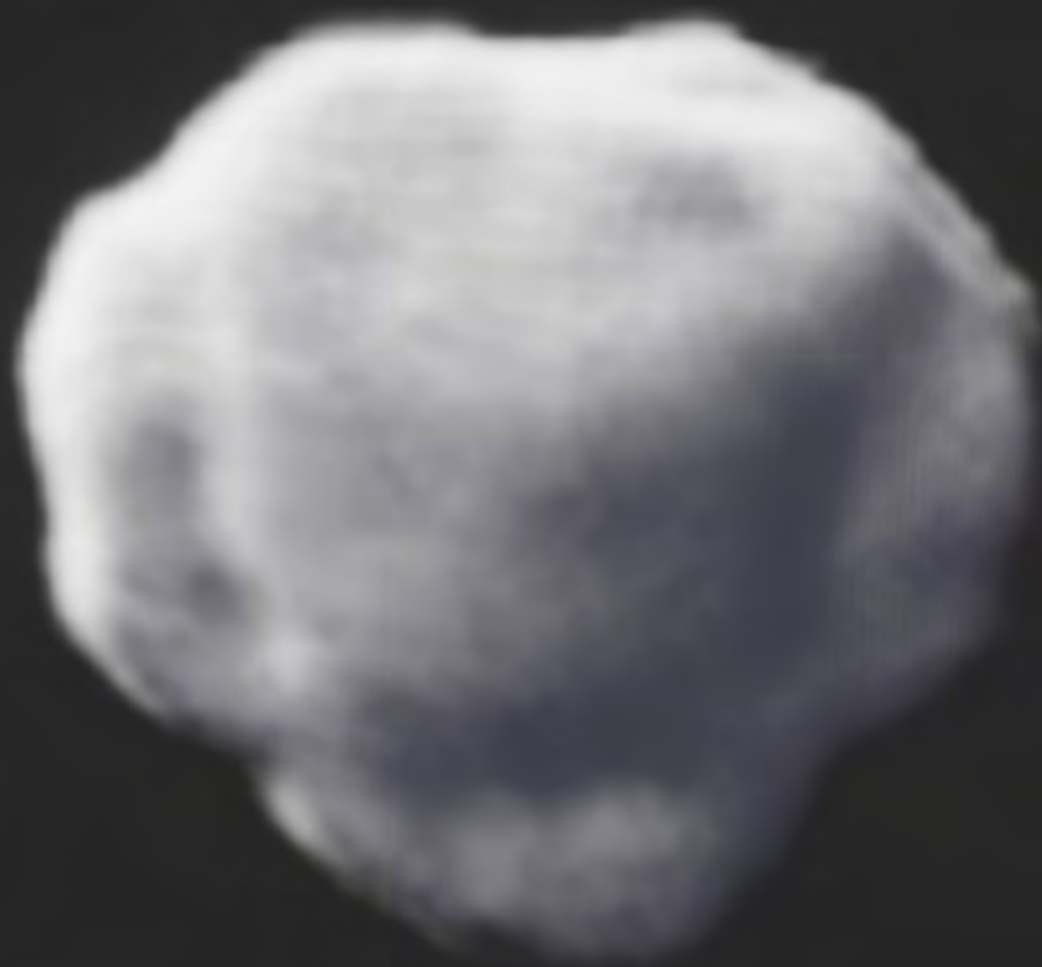


Abbildung 2: Objektmaterial von außen betrachtet (Quelle: Eigene Darstellung)



Anmerkung: Zur besseren Veranschaulichung enthält
dieser Shader auch eine simple Lichtberechnung

Abbildung 3: Objektmaterial mit Kamera an der Kante des Würfels (Quelle: Eigene Darstellung)

2.6 Zwischenfazit

Zusammenfassend zeigt die historische Entwicklung, dass sich Ray Marching von einer reinen mathematischen Oberflächenbeschreibung zu einer leistungsfähigen Echtzeit-Rendering-Technik weiterentwickelt hat. Für die praktische Umsetzung in Unity bietet das Verfahren durch die Nutzung von 3D-Texturen und Compute Shadern eine effiziente Grundlage. Auch der Dungeon ist implizit beschriebene Geometrie, wie sie bei der geschichtlichen Entwicklung des Ray Marchings umgesetzt wurde. Die architektonische Entscheidung zugunsten eines Fullscreen-Shaders löst das Problem der Kameraperspektive innerhalb eines geschlossenen Volumens und ermöglicht eine durchgängige Strahlverfolgung. Diese technische Basis bildet nun die Voraussetzung für die visuelle Umsetzung und Shader-Implementierung, die im folgenden Kapitel behandelt werden.

3 Grundlegende Implementierung

3.1 Einleitung

Aufbauend auf den theoretischen Grundlagen und der architektonischen Entscheidung für einen Fullscreen-Shader widmet sich dieses Kapitel den Grundlagen für die visuelle Umsetzung des Dungeons. Der Fokus liegt auf der Struktur des Ray Marching Shaders und der performanten Abfrage der 3D-Textur.

3.2 Shader-Aufbau

Der Shader Graph in Unity übernimmt grundsätzlich keine wichtigen Berechnungsschritte für die Darstellung des Ray Marchings. Viel mehr dient er der Zuordnung der Materialattribute, des Weitergebens dieser Attribute an die Ray Marching Funktion aus dem HLSL Code und der Flexibilitätserhaltung für zukünftige Änderungen. Die Ray Marching Funktion hat folgenden Aufbau:

```
Attribute (Kamera Position, World Position, Volumentexturen, Bereichseingrenzungen etc.)
Schleife zum Finden des relevanten Bereichs mittels Box-SDF
    Schleife zum Durchlaufen des Bereichs, bis Dichte >= 1
oder Verlassen des Bereichs
    Wenn Dichte >= 1:
        Normalenapproximation
        Triplanar Projection
        Ambient Occlusion
        Schleife zur Lichtberechnung für Directional Light
        Schleife zur Lichtberechnung für Additional Light
Rückgabe der berechneten Farbe des Bildpixels
```

Bei allen Schleifen handelt es sich um solche, bei denen der aktuell betrachtete Punkt im dreidimensionalen Raum wiederholt entlang eines Strahls nach vorn gesetzt und entweder Distanzen für den nächsten Schritt berechnet werden oder die 3D-Textur gesampelt wird.

3.3 SDF-Bereich

Die Vermeidung unnötiger Berechnungen stellt ein zentrales Optimierungsziel dar. Das effektive Ray Marching wird daher auf einen würfelförmigen Bereich (in manchen Beispielen der Einfachheit halber auch sphärisch) beschränkt, der den Dungeon-Abschnitt umfasst. Basierend auf dem Prinzip der Signed Distance Functions kann jeder Kamerastrahlschritt immer um die kürzeste Distanz zum nächsten Objekt verschoben werden, ohne dabei in Geometrien einzudringen. Diese Eigenschaft wird genutzt, um performant zu prüfen, ob sich ein Strahl überhaupt im relevanten Dungeon-Bereich befindet. Verläuft der Strahl außerhalb des Volumens, werden keine aufwendigeren Berechnungen durchgeführt. Innerhalb des Dungeons wird der Strahl schrittweise vorangetrieben und sampelt dabei eine 3D-Textur, die die räumliche Struktur des Dungeons abbildet.

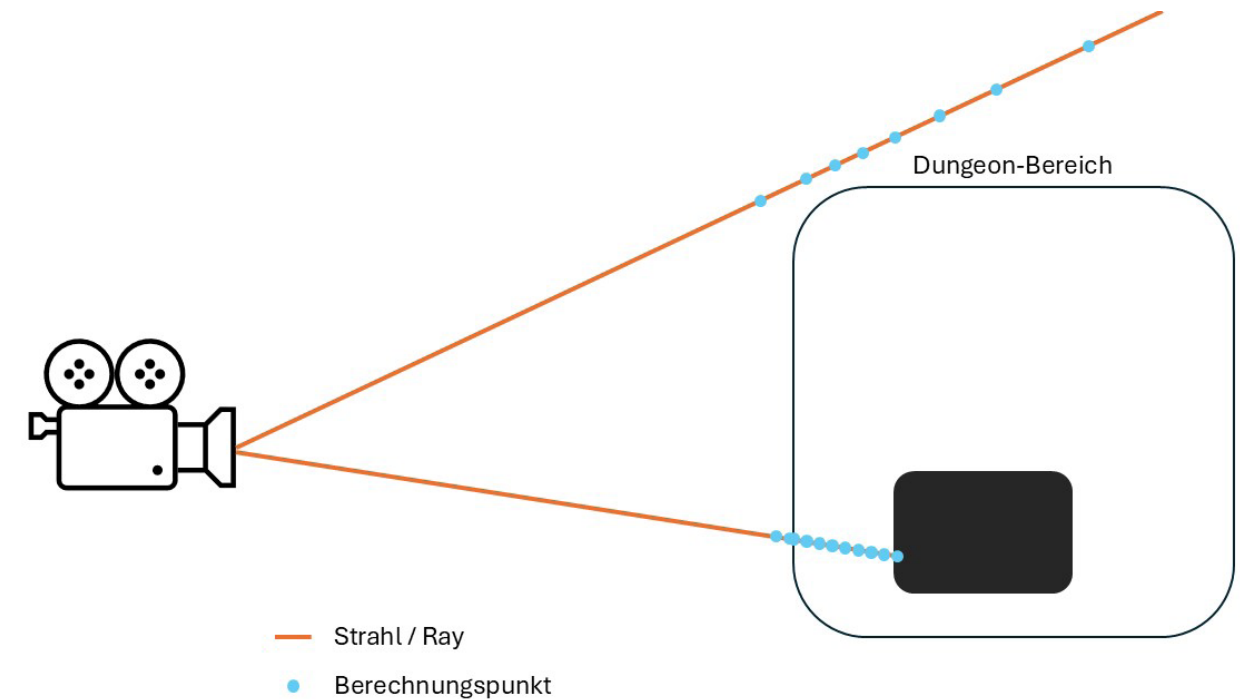


Abbildung 4: Ray Marching mit Box-SDF zum Finden des Dungeon-Bereichs, feste Schrittweiten im Dungeon-Bereich
(Quelle: Eigene Darstellung)

3.4 Dichtewerte

Der Dungeon wird zunächst als Volumen und nicht als starre Geometrie behandelt. Die gesampelten Dichtewerte werden aufaddiert. Erreicht die kumulierte Dichte den Wert 1, wird das Pixel als vollständig undurchsichtig markiert. Sobald ein Pixel als vollständig undurchsichtig markiert ist, werden weitere Berechnungen, wie in 3.2 beschrieben, durchgeführt. Ein Pixel, der keinen hohen Dichtewert erreicht, bleibt für das Ray Marching undurchsichtig und es wird die ursprüngliche Pixelfarbe ausgegeben. Die Summation der Dichte schafft im Vergleich zur sofortigen Berechnung, wenn ein gesampelter Wert > 0 ist, einen weichen Übergang und keine so harten Kanten.

3.5 Erstellung von 3D-Texturen

Zur Generierung einer Test-3D-Textur wird ein Workflow verwendet, bei dem in Blender ein Objekt innerhalb eines Würfelvolumens modelliert, in ein Volume umgewandelt und als einzelne Schichten gerendert wird. Diese Schichten werden anschließend zu einer 3D-Textur zusammengeführt.

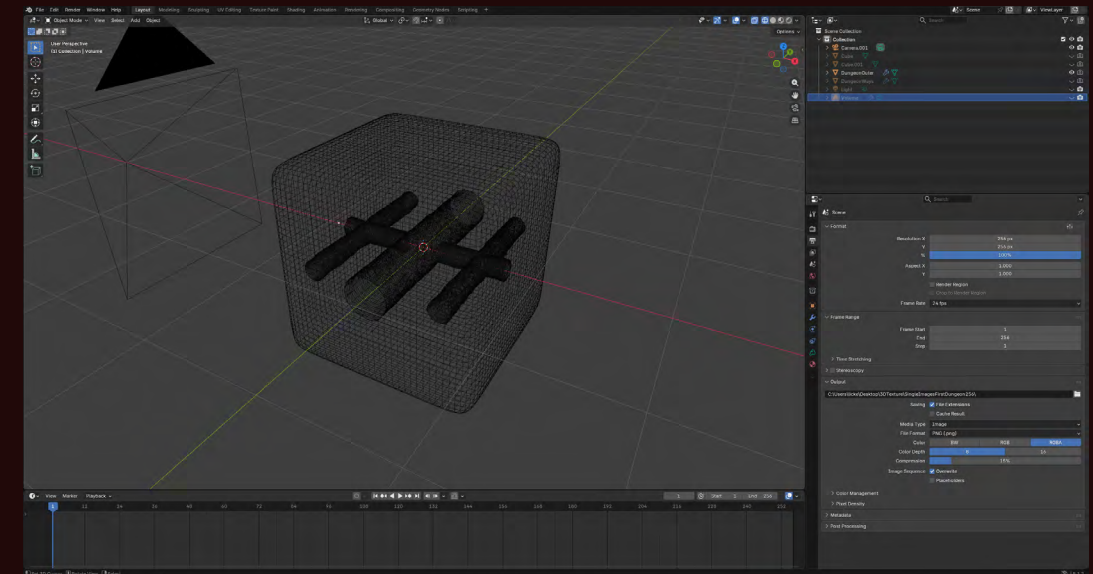


Abbildung 5: In Blender modelliertes Test-Objekt (Quelle: Eigene Darstellung)

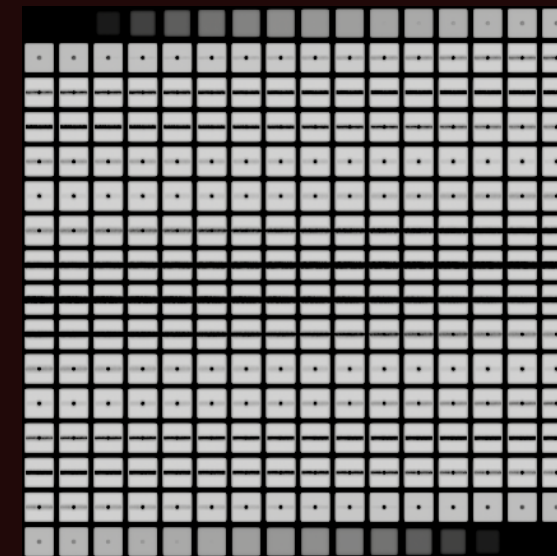


Abbildung 6: Aneinandergereihte Volumen-Slices (Quelle: Eigene Darstellung)

3.6 Zwischenfazit

Die grundlegende Implementierung legt das technische Grundgerüst für die volumetrische Darstellung des Dungeons. Durch die Kombination aus Shader Graph und Custom Functions lässt sich die komplexe Strahlverfolgung strukturiert, performant und erweiterbar umsetzen. Die Begrenzung des Berechnungsbereichs mittels SDF gewährleistet weitere Effizienz. Der beschriebene Workflow zur Erstellung von 3D-Texturen bietet eine praktikable und reproduzierbare Methode zur Generierung der zugrundeliegenden 3D-Texturen. Diese Basis wird in dem folgenden Kapitel zur visuellen Darstellung konkret mit Beleuchtung, Texturierung, Tiefenintegration und Artefaktvermeidung erweitert.

4 Visuelle Darstellung

4.1 Einleitung

Auf der Grundlage der in Kapitel 3 entwickelten Shader-Struktur widmet sich dieses Kapitel der visuellen Verfeinerung und der nahtlosen Einbindung des Ray Marching Verfahrens in die Rendering-Pipeline von Unity. Das Kapitel beginnt mit der Behandlung der Vermeidung von Schrittwertenartefakten und der Implementierung einer Tiefenprüfung anhand des Unity-Depth-Buffers. Anschließend werden die Integration von homogenem Umgebungslicht, Directional Light und Point Light detailliert beschrieben. Im weiteren Verlauf folgen die Techniken zur UV-unabhängigen Texturierung mittels Triplanar Projection, die Differenzen-Approximation von Normalenvektoren sowie ein Ambient Occlusion Effekt zur Steigerung der räumlichen Tiefe. Diese Verfahren schließen die Lücke einer reinen Volumenabtastung und einer visuell überzeugenden, spielgerechten Darstellung.

4.2 Vermeidung von Artefakten

Die schrittweise Strahlverfolgung führt zu sichtbaren Stufenartefakten, da benachbarte Pixel identische Schrittweiten aufweisen. Auch wenn diese Artefakte bei einer späteren Texturierung des Dungeons vermutlich weniger auffallen würden, ist es sinnvoll, derartige Probleme frühzeitig anzugehen.

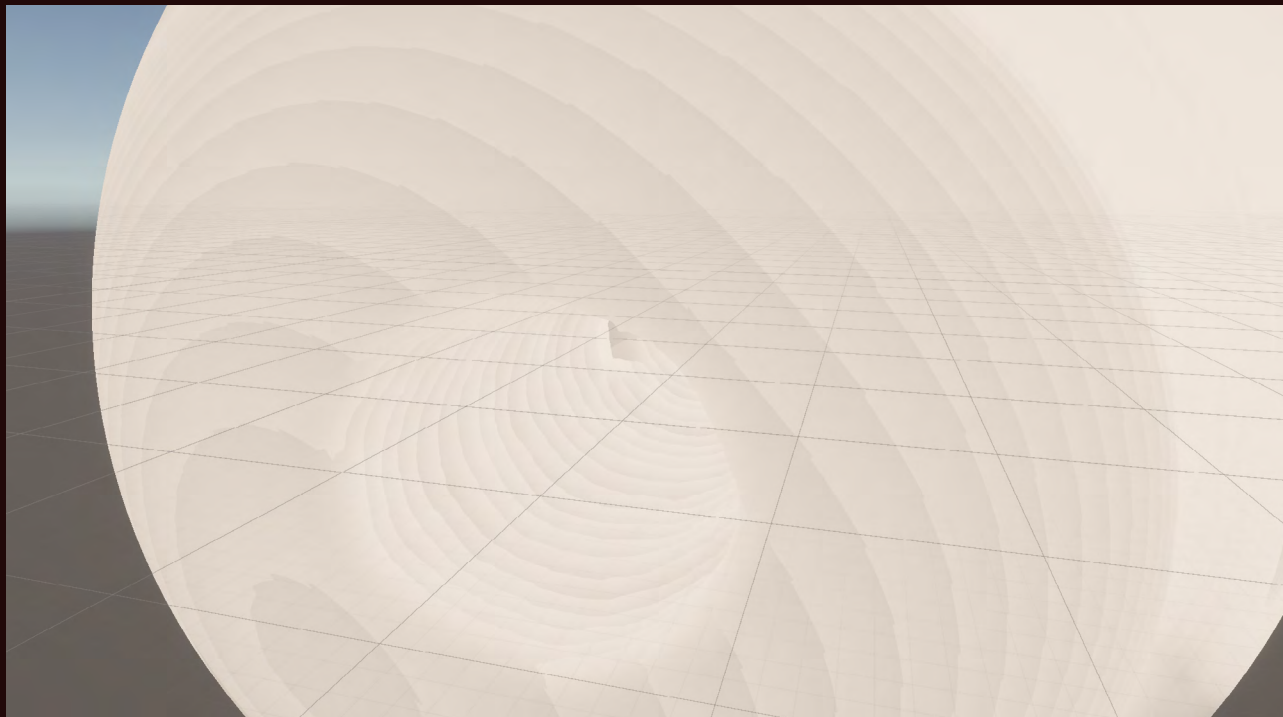


Abbildung 7: Sichtbare Streifen-Artefakte (Quelle: Eigene Darstellung)

Zur Beseitigung dieses Effekts wird eine zeitlich variable Noise-Textur eingesetzt. Ein pseudozufälliger Wert aus dieser Textur wird mit einem gewichteten Faktor zur Schrittweite des ersten Strahlschritts addiert. Dies erzeugt einen kontrollierten Versatz an den Übergängen, der die harten Abstufungen visuell auflöst und einen weichen Blend-Effekt erzeugt.

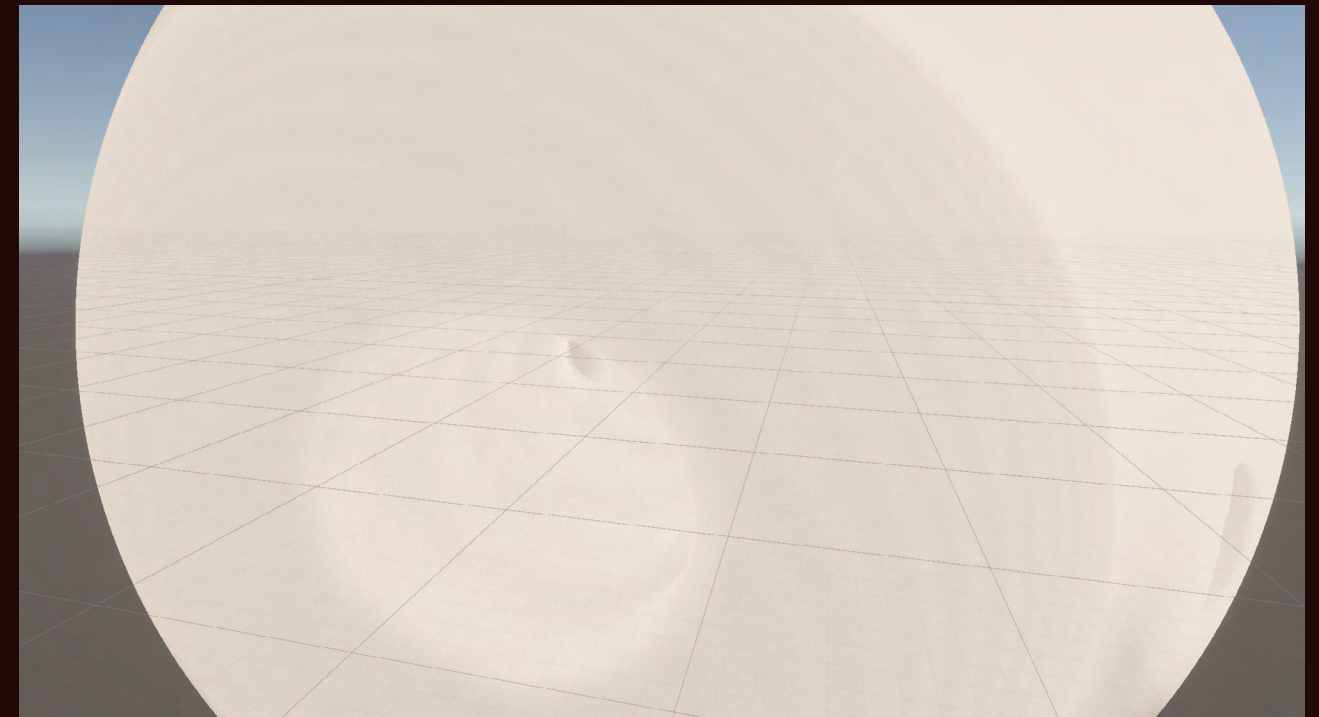


Abbildung 8: Abstufungs-Optimierung (Quelle: Eigene Darstellung)

4.3 Tiefenprüfung

Im Gegensatz zu polygonalen Objekten, bei denen der Depth Buffer automatisch die Sichtbarkeitsreihenfolge bestimmt, muss die Tiefenintegration bei einem Ray-Marching-Shader explizit implementiert werden. Um polygonale Objekte korrekt in die volumetrische Darstellung zu integrieren, wird der bereits von Unity befüllte Depth Buffer ausgelesen. Die Position der Kamera und die aktuelle Strahlposition sind bekannt.

Übersteigt die Strahllänge den im Depth Buffer gespeicherten Tiefenwert, wird die Ray-Marching-Berechnung für dieses Pixel abgebrochen. Das Pixel behält entweder seinen ursprünglichen Zustand oder wird nur mit der bis dahin berechneten Dichte modifiziert.

Die Werte des Depth-Buffers sind keine linearen Werte, lassen sich aber durch eine integrierte Funktion umrechnen. Dadurch können polygonale Objekte, die nicht transparent sind, also welche mit einem Opaque Material, korrekt dargestellt werden.

Auch transparente Objekte in der Tiefenreihenfolge korrekt zu berücksichtigen, ist für diese Arbeit nicht vorgesehen. Transparente Objekte werden in der Rendering Pipeline nach dem Fullscreen Ray Marching dargestellt und erscheinen somit immer vor dem Ray Marching. Eine Anwendung davon ist das Glas der Lampe unter 7.3.1.

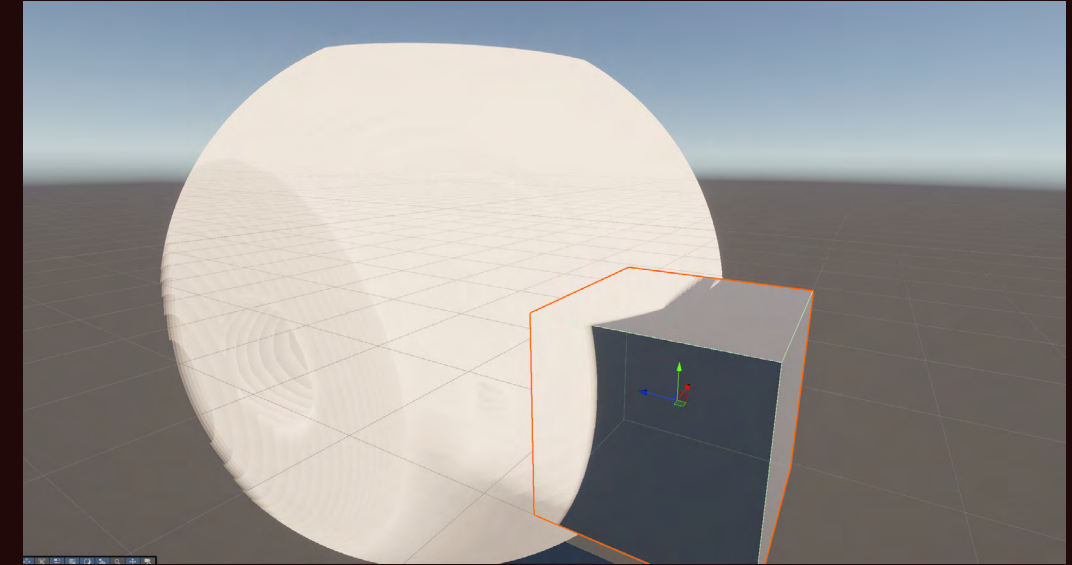


Abbildung 9: Ein sich teilweise im Ray Marching befindender Würfel ohne Tiefenprüfung (Quelle: Eigene Darstellung)

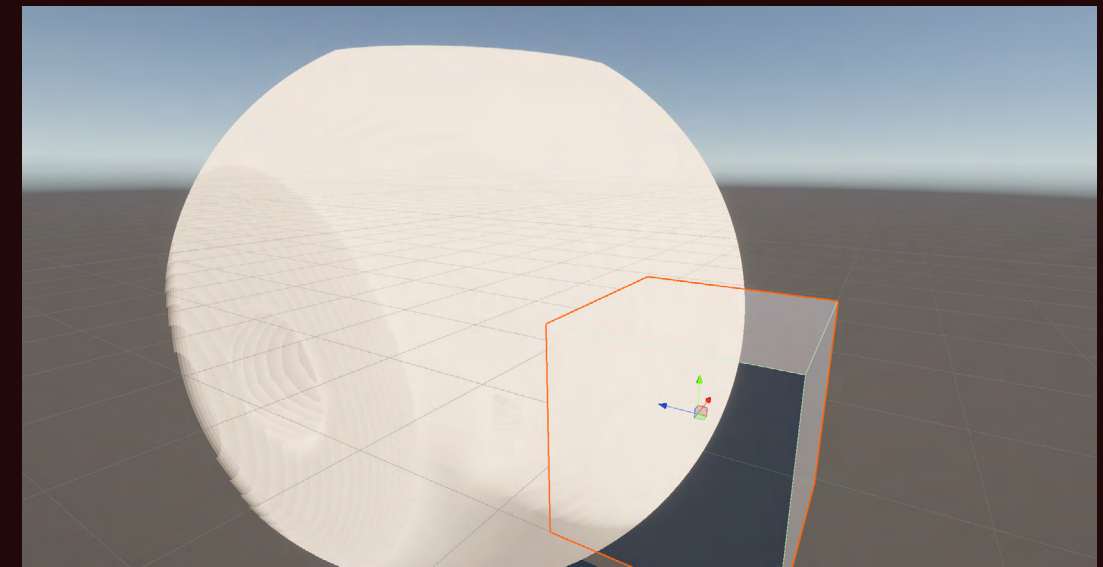


Abbildung 10: Ein sich teilweise im Ray Marching befindender Würfel mit Tiefenprüfung (Quelle: Eigene Darstellung)

4.4 Beleuchtung

4.4.1 Homogene Beleuchtung

In Unity wird auf alle Materialien, selbst ohne in der Szene platzierte Lichtquellen, standardmäßig eine Beleuchtung angenommen, die entweder durch die Farben der Skybox oder durch eine festgelegte Ambient Light Farbe dafür sorgt, dass kein Material komplett schwarz ist.

Im Ray Marching fällt die Berechnung von Unity zwar weg, aber sie ist sehr einfach selber durchführbar. Es wird eine Farbe festgelegt, in diesem Fall ein dunkles grau, das zudem einen geringen blau-Anteil hat und diese Farbe wird als Basis für die Lichtberechnung genommen für Stellen, die nicht von Lichtquellen beleuchtet sind.

4.4.2 Directional Light

Beleuchtung des Dungeons

Ein Directional Light agiert in der Szene wie eine Sonne. Es hat keine Position sondern strahlt aus einer Richtung gleichermaßen die ganze Szene an.

Zur Integration des Directional Lights wird der Shader Graph verwendet. Dieser bietet den Get-MainLightDirection Node. Diese Richtung wird an den Ray Marching Shader weitergegeben. Für die Lichtfarbe bietet Unity in Shadern eine eingebaute Funktion.

An der Stelle, an der der ursprüngliche Strahl des Ray Marchings die Dichtegrenze von 1 erreicht, wird ein sekundärer Strahl in Richtung der Sonne, bzw. die umgekehrte Richtung des Directional Lights initiiert. Entlang dieses Lichtstrahls wird die 3D-Textur abgetastet und die Dichte akkumuliert. Bereiche mit hoher Dichte werfen Schatten, während Bereiche mit niedriger oder null Dichte vom Directional Light beleuchtet werden. Für weiche Übergänge zwischen Licht und Schatten wird eine lineare Interpolation (lerp) eingesetzt.

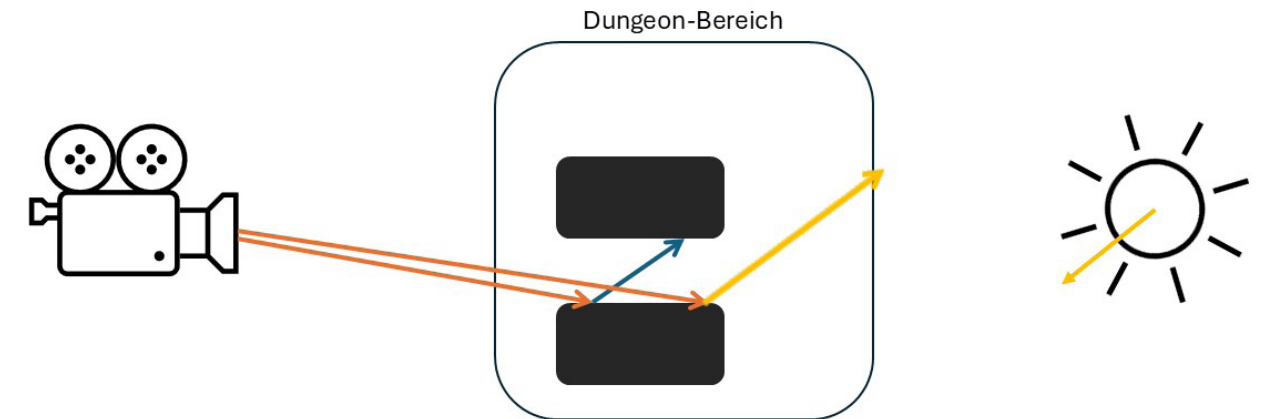


Abbildung 11: Beleuchtung des Dungeons durch Directional Light (Quelle: Eigene Darstellung)

Beleuchtung von Polygon-Objekten im Dungeon

Für polygonale Objekte innerhalb des Dungeons wird ein separater Lit-Shader Graph implementiert. Dieser enthält einen Custom Function Node, der im Fragment Shader pro Pixel eine vereinfachte Ray-Marching-Berechnung ausführt. Das Ergebnis ist ein Skalarwert zwischen 0 und 1, der angibt, inwieweit das Pixel vom Dungeon beschattet wird.

Der Fullscreen-Shader berechnet zusätzlich die ShadowAttenuation des Directional Lights, wodurch auch die von Unity berechneten Echtzeit-Schatten polygonaler Objekte korrekt auf das volumetrische Modell projiziert werden.



Abbildung 12: Schatten von Polygon-Objekt auf dem Dungeon, Schatten vom Dungeon auf dem Polygon-Objekt
(Quelle: Eigene Darstellung)

4.4.3 Point Light

In geschlossenen Dungeon-Bereichen dringt direktes Sonnenlicht selten ein. Daher wird eine tragbare Lichtquelle des Spielers, beispielsweise eine Fackel, als Point Light implementiert. In Unity zählen Point Lights zu den Additional Lights, auf die Fullscreen-Shader standardmäßig keinen direkten Zugriff haben, da Additional Lights nur an Objekte gesendet werden, die sich in deren Einflussbereich befinden (vgl. Unity Technologies, 2026e). Fullscreen-Shader können jedoch auf das dem Kamera-Transform nächstgelegene Additional Light zugreifen. Diese Einschränkung wird akzeptiert und die Berechnung entsprechend angepasst.

Die Lichtfarbe, die distanzbasierte Intensität und Schatteninformationen von Polygon-Objekten sind direkt in den Lichtdaten enthalten. Für die Schattenberechnung des Dungeons wird die Position des Point Lights benötigt, die in den Standard-Lichtdaten nicht enthalten ist. Daher wird die Position als Laufzeit-Variable bereitgestellt und synchron mit der Position des Lampen-Objekts aktualisiert. Die Berechnung verläuft analog zur Directional-Light-Integration. Die bereits in den Lichtdaten enthaltene Intensitätskurve ermöglicht eine direkte Kopplung an dynamische Effekte, wie einen flackernen Lampen-Shader.



Abbildung 13: Point Light Einfluss auf die Beleuchtung des Dungeons (Quelle: Eigene Darstellung)

4.5 Texturierung

4.5.1 Triplanar Projection

Die Texturierung polygonaler Modelle erfolgt üblicherweise über UV-Mapping. Dieses Verfahren ist für volumetrisches Ray Marching nicht anwendbar. Stattdessen kommt Triplanar Projection zum Einsatz. Die Unity-Dokumentation (vgl. Unity Technologies, 2023a) stellt den zugrundeliegenden Algorithmus bereit, der die Position und die Normalenvektoren nutzt, um drei orthogonale Texturen gewichtet zu kombinieren:

```
float3 Node_UV = Position * Tile;  
float3 Node_Blend = pow(abs(Normal), Blend);  
Node_Blend /= dot(Node_Blend, 1.0);  
float4 Node_X = SAMPLE_TEXTURE2D(Texture, Sampler, Node_UV.zy);  
float4 Node_Y = SAMPLE_TEXTURE2D(Texture, Sampler, Node_UV.xz);  
float4 Node_Z = SAMPLE_TEXTURE2D(Texture, Sampler, Node_UV.xy);  
float4 Out = Node_X * Node_Blend.x + Node_Y * Node_Blend.y + Node_Z * Node_Blend.z;
```

4.5.2 Normalenapproximation

Während die Strahlposition direkt verfügbar ist, existieren bei volumetrischen Daten keine expliziten Normalen. Diese werden stattdessen durch eine Differenzen-Approximation berechnet. Dazu wird die 3D-Textur in alle sechs Hauptachsenrichtungen um einen kleinen Offset abgetastet. Die Differenz der Dichtewerte ergibt einen Vektor, der zur Normalen approximiert wird:

```
float up = SAMPLE_TEXTURE3D(volumeTex, volumeSampler, frac((rayPosition + float3(0, distanceToCheck, 0)) * volumeDensity)).r;  
float down = SAMPLE_TEXTURE3D(volumeTex, volumeSampler, frac((rayPosition + float3(0, -distanceToCheck, 0)) * volumeDensity)).r;  
float left = SAMPLE_TEXTURE3D(volumeTex, volumeSampler, frac((rayPosition + float3(-distanceToCheck, 0, 0)) * volumeDensity)).r;  
float right = SAMPLE_TEXTURE3D(volumeTex, volumeSampler, frac((rayPosition + float3(distanceToCheck, 0, 0)) * volumeDensity)).r;  
float forward = SAMPLE_TEXTURE3D(volumeTex, volumeSampler, frac((rayPosition + float3(0, 0, distanceToCheck)) * volumeDensity)).r;  
float backward = SAMPLE_TEXTURE3D(volumeTex, volumeSampler, frac((rayPosition + float3(0, 0, -distanceToCheck)) * volumeDensity)).r;  
float3 approxNormal = normalize(float3(right - left, up - down, forward - backward));
```

Dabei sei aber angemerkt, dass es sich bei diesen Normalen lediglich um Schätzwerte handelt und nicht um genaue um genaue Werte.

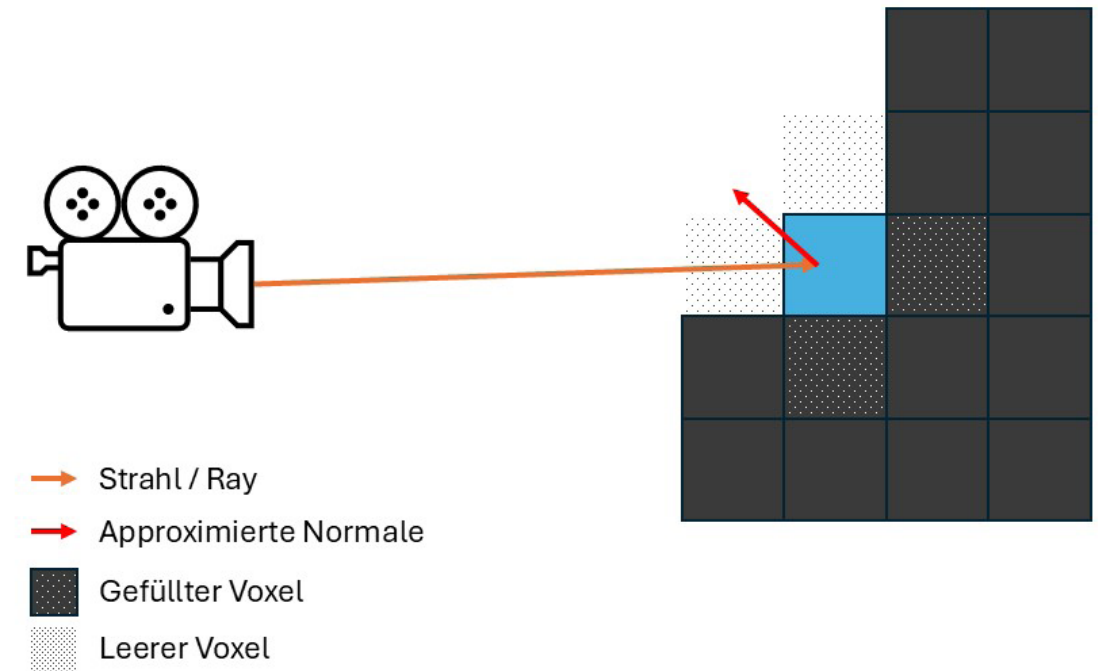


Abbildung 14: Schematische Darstellung der Normalenapproximation (Quelle: Eigene Darstellung)

4.5.3 Ausnahmenbehandlung

Für Randfälle, bei denen alle Nachbarn gefüllt oder leer sind und ein Nullvektor resultiert, wird die Normale auf $(0, 1, 0)$ festgelegt. Die approximierten Normalen ermöglichen eine korrekte Gewichtung der Triplanar Projection und erzeugen ein visuell kohärentes Texturierungsergebnis.

4.5.4 Farbtextur

Unter Nutzung der approximierten Normalen lässt sich nun die Triplanar Projection aus 4.5.1 anwenden. Den Pixeln auf dem Bildschirm, die durch Ray Marching beeinflusst werden, wird unabhängig von jeglichen UV Koordinaten nur anhand der Position und der errechneten Normalen eine Farbe aus einer beliebigen Textur zugewiesen.

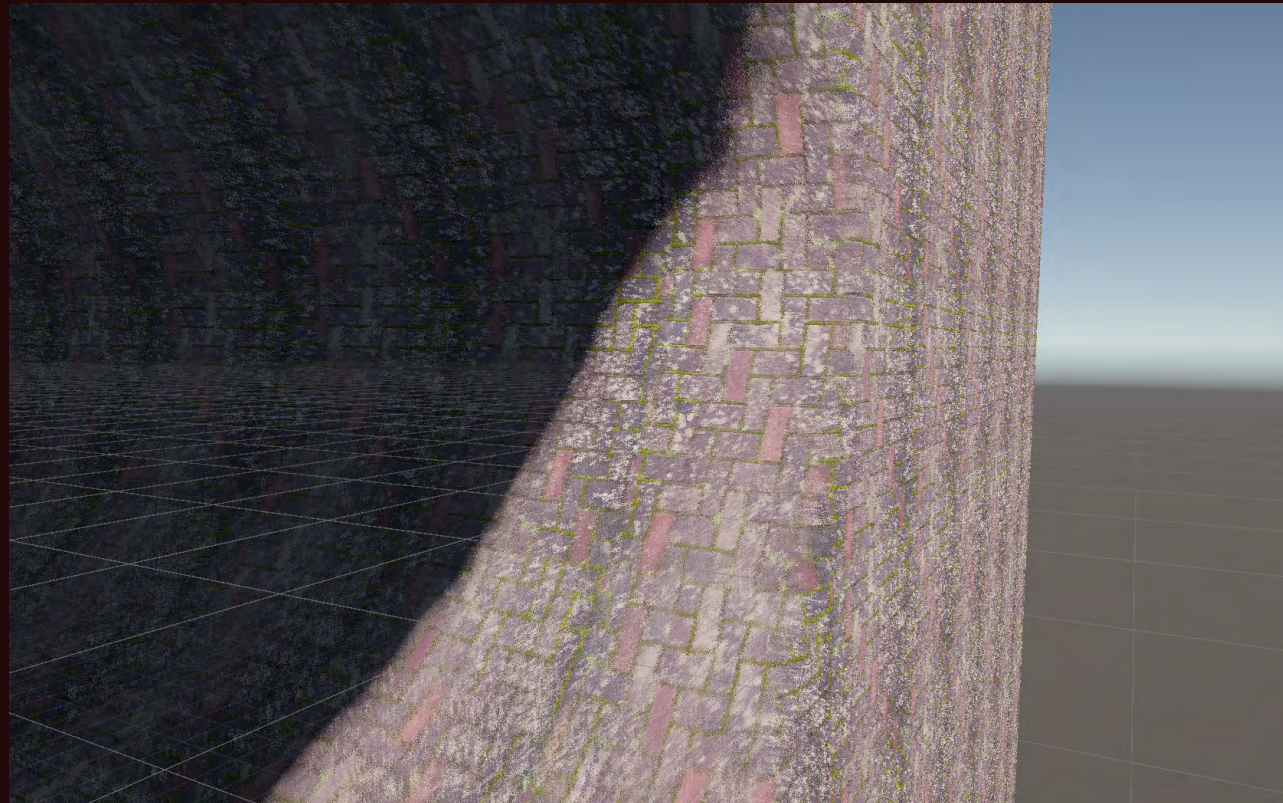


Abbildung 15: Texturierung mittels triplanar projection (Quelle: Eigene Darstellung)



Abbildung 16: Einfluss der Ambient Occlusion Map (Quelle: Eigene Darstellung)

4.5.5 Ambient Occlusion Textur

Die ausschließliche Verwendung von Farbtexturen führt zu einer optisch flachen Darstellung. Normalerweise ist die Verwendung von Normal Maps der Standard, um Oberflächen mehr Tiefe zu verleihen. Dabei wird die Lichtberechnung mithilfe einer Textur beeinflusst, um Erhebungen und Vertiefungen darzustellen. Die approximierten Normalen sind dafür nicht präzise genug.

Um die Tiefenwahrnehmung anderweitig zu verbessern, kommt eine Ambient Occlusion Map zum Einsatz. Dabei handelt es sich um eine Graustufen-Textur. Wie die Farbtextur, wird sie auch triplanar projiziert. Bei der Lichtberechnung wird die Stärke der Beleuchtung vom Wert der Ambient Occlusion Textur an der Stelle abhängig gemacht. Es werden nur Stellen stark beleuchtet, die sich nicht in Materialvertiefungen befinden. Zusätzlich wird auch in Bereichen die im Schatten liegen, eine Verdunkelung der Vertiefungen mit der Ambient Occlusion Textur durchgeführt. Dadurch entsteht eine glaubhafte Illusion von Materialtiefe.

4.6 Erweiterte Schattierung

Die zur Normalenapproximation ermittelten Dichtewerte in sechs Richtungen werden genutzt, um die lokale Voxel-Dichte zu bestimmen. Für eine noch etwas höherwertiges Ergebnis werden noch zusätzliche Werte angrenzender Voxel mit einbezogen. Hohe Dichtewerte führen zu einer gezielten Verdunkelung, was den Ambient Occlusion-Effekt physikalisch plausibel unterstützt.

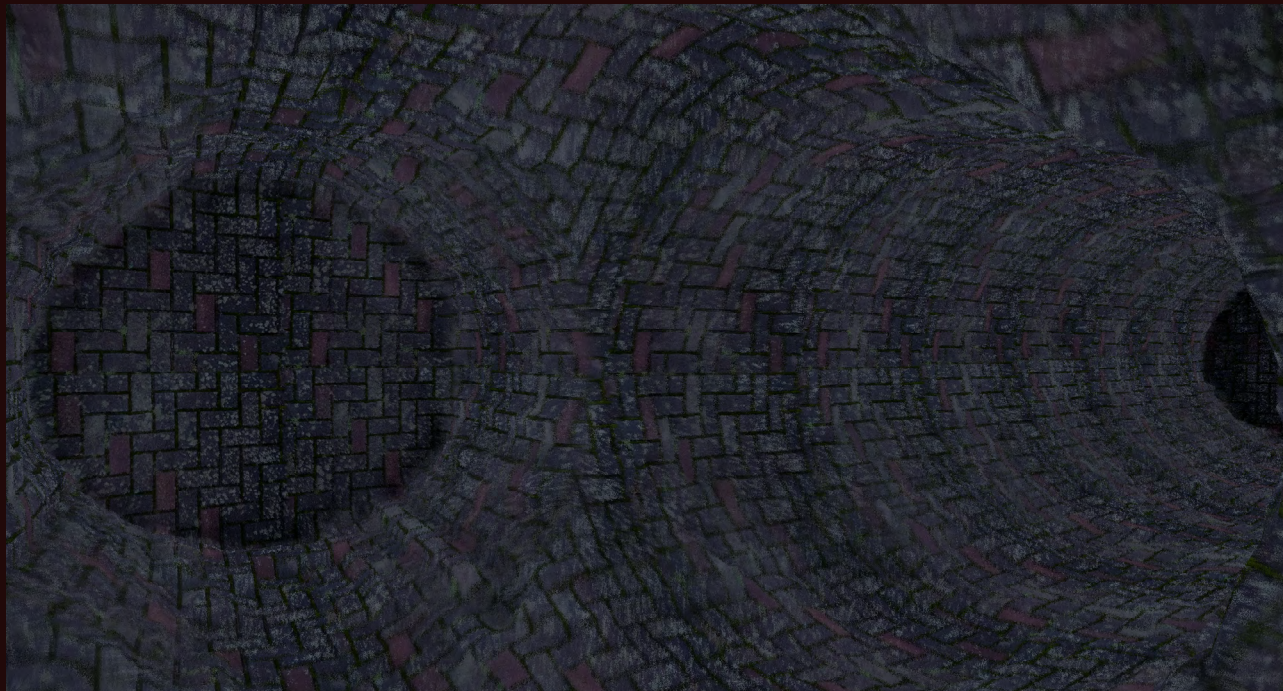


Abbildung 17: Verdunkelung von Stellen mit dicht beieinanderliegenden Voxeln (Ambient Occlusion)
(Quelle: Eigene Darstellung)

4.7 Zwischenfazit

Die Shader-Implementierung demonstriert, wie Ray Marching durch gezielte Optimierungen und Shader-Techniken wie Triplanar Projection, Differenzen-Normalen und Ambient Occlusion visuell überzeugende Ergebnisse liefert. Die Integration von Directional und Point Lights, die korrekte Tiefenprüfung mit polygonalen Objekten sowie die Vermeidung von Schrittweitenartefakten gewährleisten eine stabile Echtzeitdarstellung. Während dieses Kapitel die visuelle Ebene abschließt, bleibt die physikalische Interaktion mit dem Volumen noch ungelöst. Die notwendige Kollisionsbehandlung und Physikintegration wird daher im nächsten Kapitel behandelt.

5 Physik

5.1 Einleitung

Ein rein visuelles Volumen ist für ein interaktives Spiel nicht ausreichend. Da Ray Marching keine explizite Mesh-Geometrie erzeugt, kann das standardmäßige Unity-Physiksystem nicht direkt auf die Dungeon-Struktur zugreifen. Dieses Kapitel beschreibt daher die Konzeption einer GPU-basierten Collider-Generierung, die es ermöglicht, physikalische Kollisionen in Echtzeit zu berechnen und in die Unity Physiksimulation zu integrieren. Zusätzlich wird die Charaktersteuerung sowie das Bewegungsfeedback erläutert, um eine immersive Spielerfahrung zu gewährleisten.

5.2 GPU-basierte Collider-Generierung

Die Implementierung von Kollisionen stellt eine zentrale Anforderung dar, da der Spieler sich durch den Dungeon bewegen, Wege freimachen und auf regenerierende Strukturen reagieren muss. Eine naive Implementierung, bei der für jeden gefüllten Voxel der 3D-Textur ein Collider erstellt wird, ist bei einer Auflösung von 256^3 Voxel (ca. 16,7 Millionen) performancetechnisch nicht tragbar. Eine bedarfsgesteuerte Neuberechnung wäre zudem komplex und würde dynamische Änderungen nicht in Echtzeit abbilden. Eine rein shaderbasierte Rückstoßberechnung wäre zwar performant, jedoch inkompatibel mit dem Unity-Physiksystem und schwer auf nicht-sphärische Objekte übertragbar.

Eine weitere denkbare Lösung wäre es, aus der 3D-Textur ein Mesh zu berechnen und dieses Mesh als einen Mesh Collider in die Physik zu integrieren. Dieser Ansatz hat sich allerdings gar nicht erst angeboten, denn es ist bereits im Vorhinein klar, dass es sich dabei um einen enormen Rechenaufwand handeln würde, der aufgrund der dynamischen Veränderbarkeit des Dungeons ständig neu durchgeführt werden muss. Es bestünde auch die Notwendigkeit CPU-seitig auf die 3D-Textur zugreifen zu müssen, was zu viel zu langen Synchronisierungs-Wartezeiten zwischen CPU und GPU führt.

Die gewählte Lösung kombiniert GPU-Berechnung mit dem Unity-Physiksystem. Ein Compute Shader liest die aktuelle 3D-Textur aus und erhält als Parameter die Position und einen Radius eines physikalischen Objekts. Innerhalb dieses Radius werden gefüllte Voxel identifiziert und deren Positionen in einen ComputeBuffer geschrieben. Dieser Shader wird in der FixedUpdate-Methode aufgerufen, die in Unity vor der Physiksimulation ausgeführt wird (vgl. Unity Technologies, 2026d), (vgl. Unity Technologies, 2026c). Dadurch stehen den Rigidbody-Berechnungen stets aktuelle Kollisionsdaten zur Verfügung. Eine Ausführung in der Update-Methode wäre redundant und performancerelevant.

Skripte auf den betroffenen Objekten automatisieren die Übergabe der Parameter an den Compute Shader. Als Kollisionsgeometrie werden SphereColliders verwendet, da sie die geringste Berechnungskomplexität aufweisen (vgl. Unity Technologies, 2026b). Die Generierung erfolgt nur im unmittelbaren Einflussbereich des Objekts, wodurch die Collider-Anzahl minimal bleibt. Die Berechnung eines MeshColliders wäre bei dieser Dynamik ineffizient. Objekte können beliebige eigene Collider nutzen; für den Spieler werden drei Berechnungspunkte (Kopf, Körpermitte, Füße) verwendet, um die Kapselgeometrie präzise abzubilden, ohne den Erfassungsradius unnötig zu vergrößern.

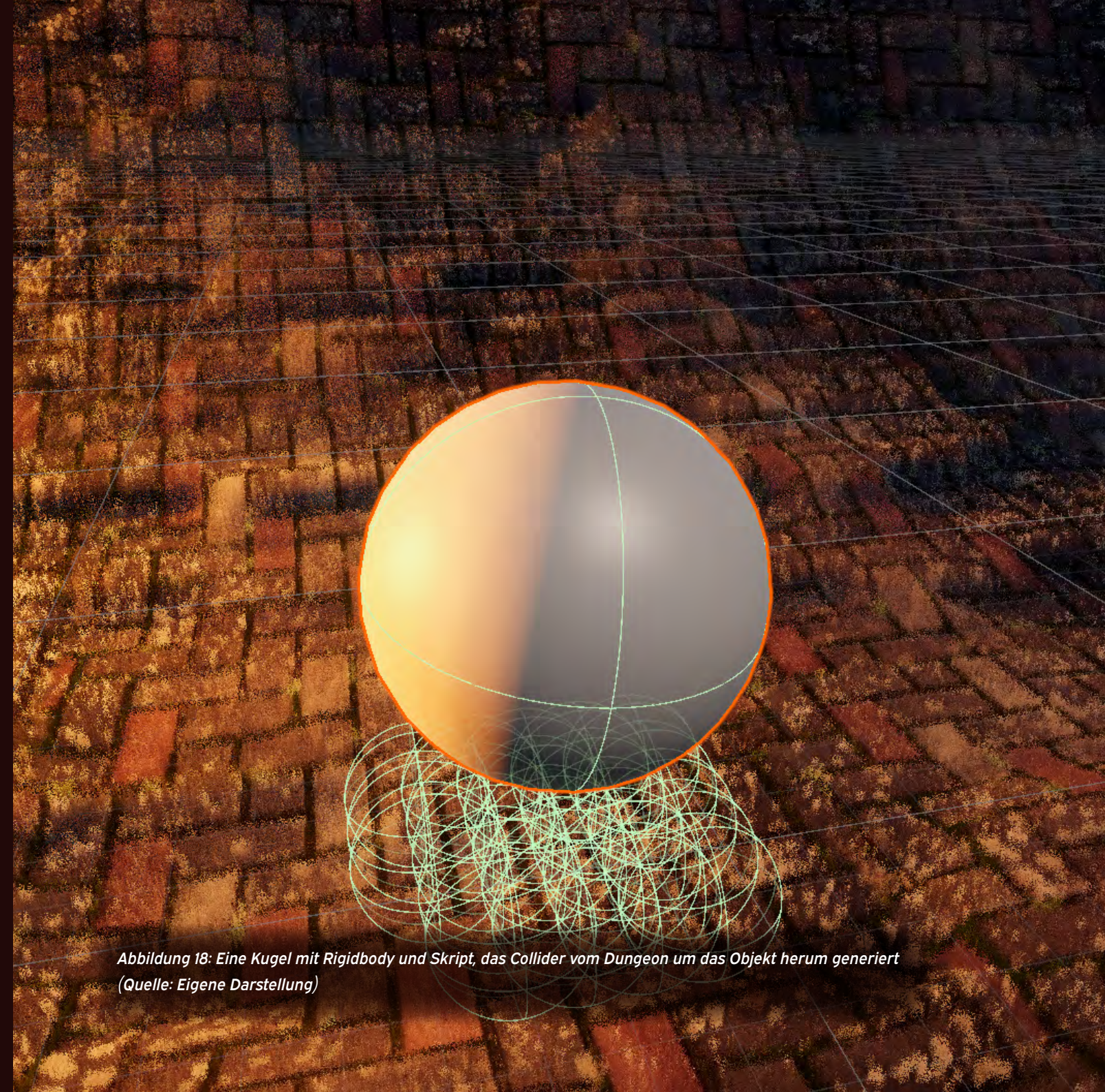


Abbildung 18: Eine Kugel mit Rigidbody und Skript, das Collider vom Dungeon um das Objekt herum generiert
(Quelle: Eigene Darstellung)

5.3 Charaktersteuerung

5.3.1 Character Controller

Zur Steuerung der Spielphysik wird ein Rigidbody-basierter Controller eingesetzt, da dieser maximale Kompatibilität mit anderen physikalischen Objekten bietet. Die Bewegung erfolgt über direkte Velocity-Modifikation. Die Kamera ist als Child-Objekt an den Spieler gekoppelt und folgt dessen Rotation und Translation. Zur Steigerung der Immersion wird die Kameraposition über eine Sinusfunktion leicht vertikal moduliert.

5.3.2 Bewegungsfeedback

Bei Bewegung erhöht sich die Frequenz dieser Modulation, was einen atmungsähnlichen Effekt simuliert. Zusätzlich skaliert das Field of View (FOV) mit der Bewegungsgeschwindigkeit, wodurch besonders bei Beschleunigungen oder Stürzen ein verstärktes Geschwindigkeitsgefühl entsteht.

5.4 Zwischenfazit

Die GPU-basierte Collider-Generierung in Kombination mit dem Rigidbody-Controller löst das fundamentale Problem der physikalischen Interaktion mit einem Volumen. Durch die radiusbasierte Erzeugung von SphereCollidern in der FixedUpdate-Methode bleibt die Performance stabil, während die Kapselgeometrie des Spielers erwartungsgemäß mit allen sichtbaren Teilen des Dungeons kollidiert. Die Kameramodulation und FOV-Anpassung ergänzen dies durch ein immersives Bewegungsfeedback. Damit ist die technische Basis für eine interaktive Umgebung geschaffen. Im folgenden Kapitel wird diese Grundlage um dynamische Veränderungen erweitert und das daraus resultierende Gameplay-Konzept vorgestellt.

6 Dynamische Veränderung

6.1 Einleitung

Die vorherigen Kapitel haben die visuelle und physikalische Grundlage des Dungeons etabliert. Um die Umgebung jedoch dynamisch und gameplayrelevant zu machen, müssen die volumetrischen Daten zur Laufzeit veränderbar sein. Dieses Kapitel beschreibt die GPU-seitige Implementierung von Abbau und Regeneration sowie die kanalabhängige Materialdifferenzierung.

6.2 GPU-basierter Abbau und Regeneration

Die Umrechnung von Weltkoordinaten in Texturkoordinaten ermöglicht gezielte Modifikationen der 3D-Textur. Eine CPU-seitige Bearbeitung führte bei jeder Änderung zu sichtbaren Performance-Einbrüchen, da die Daten zwischen CPU und GPU transferiert werden mussten. Daher wird ein dedizierter Compute Shader für den Dungeon-Abbau implementiert. Dieser erhält die Abbau-Position (in Texturkoordinaten) und einen Radius und modifiziert die 3D-Textur direkt auf der GPU. Der Performance-Ruckler entfällt vollständig.

Die Regeneration erfolgt kontinuierlich und wird ebenfalls vollständig auf der GPU verarbeitet. Ein separater Compute Shader erhält die aktuelle und die ursprüngliche 3D-Textur. In jedem Fixe-Update-Zyklus werden die Werte der aktuellen Textur schrittweise an die Referenztextur angepasst, wodurch ein organisches Nachwachsen simuliert wird.

6.3 Materialdifferenzierung

Zur Unterscheidung verschiedener Materialien wird eine zusätzliche 3D-Textur eingeführt, die alle RGB-Kanäle nutzt. Im Ray-Marching-Shader werden Farb- und Ambient Occlusion-Texturen kanalabhängig zugeordnet: Rot entspricht Material 1, Grün Material 2, Blau Material 3. Der Alphakanal steht für ein mögliches viertes Material zur Verfügung. Diese Materialtextur wird an die Compute Shader für Abbau und Regeneration übergeben. Dadurch lassen sich abbauabhängige Widerstandswerte und unterschiedliche Regenerationsgeschwindigkeiten pro Materialkanal definieren.



Abbildung 19: Teilweise abgebauter Dungeon mit verschiedenen Materialien (Quelle: Eigene Darstellung)

6.4 Zwischenfazit

Die Implementierung GPU-basierter Compute Shader hat es ermöglicht, die volumetrischen Daten des Dungeons zur Laufzeit effizient und ohne nennenswerte Performanceeinbußen zu modifizieren. Durch die Trennung von Abbau und Regeneration in separate Shader sowie die kanalabhängige Materialdifferenzierung lässt sich ein organisch wirkendes, sich selbst erneuerndes Umgebungsmodell realisieren. Diese technische Flexibilität bildet die Voraussetzung für ein dynamisches Gameplay, bei dem die Umgebung auf Spieleraktionen reagiert. Damit ist die Grundlage für die spielerische Nutzung des Dungeons gelegt, die im folgenden Kapitel mit Spielkonzept und Ausrüstungsgegenständen vorgestellt wird.

7 Gameplay

7.1 Einleitung

Dieses Kapitel widmet sich der spielerischen Nutzung der in den vorigen Kapiteln entwickelten Technologien. Die technische Grundlage des Ray Marching mit der GPU-basierten Volumenmodifikation wird nun in ein Gameplay-Konzept überführt. Dabei steht die Exploration eines sich dynamisch verändernden Raums im Mittelpunkt, bei dem sich der Spieler durch gezielte Interaktion Wege bereitet, sich orientieren muss und ein Ziel erreicht. Das Kapitel gliedert sich in die zunächst in die Vorstellung des übergeordneten Spielekonzepts, gefolgt von einer detaillierteren Beschreibung der Ausrüstungsgegenstände. Diese Instrumente verbinden technische Shaderlösungen mit gameplayrelevanten Mechaniken und demonstrieren, wie volumetrische Daten nicht nur dargestellt, sondern aktiv in die Spielerfahrung integriert werden können.

7.2 Konzept

Der Spieler startet in einem begrenzten Raum, umgeben von Dungeon-Strukturen. Die Aufgabe besteht in der Erforschung der Umgebung. Zur Wegbereitung stehen eine Lampe und Granaten zur Verfügung. Der Dungeon weist eine regenerative Eigenschaft auf: Geschädigte Bereiche schließen sich wie bei einem lebenden Organismus wieder. Diese Regeneration verläuft jedoch nicht uniform. An manchen Stellen wächst die Struktur nach, an anderen bleibt die Öffnung stabil. Bestimmte Bereiche weisen eine so hohe Widerstandsfähigkeit auf, dass sie durch Granaten nicht durchbrochen werden können.

Die Materialverteilung erzwingt einen labyrinthartigen Fortschritt. Ein direkter Weg zum versteckten Schatz ist nicht möglich. Der Spieler muss alternative Routen erkunden. Neben dem Hauptschatz existieren weitere Gegenstände. Ein Schatzweiser signalisiert durch schnelles Blinken und akustische Signale die Richtung zum Ziel. Ein Dichtescanner warnt vor nicht zerstörbaren Wandstrukturen. Alle Instrumente werden dem Spieler bereits zu Beginn der Expedition zur Verfügung gestellt, um die Orientierung zu unterstützen und dem Spiel ein Ziel aufzuzeigen.

Granaten stehen unbegrenzt zur Verfügung, unterliegen jedoch einer Wurf- und Zündverzögerung. Ein Fernzünder ermöglicht die sofortige Detonation aller aktiven Granaten.

Das Konzept ist bewusst passend zur Demonstration der verwendeten Technologie gewählt. Mit den Granaten als Hauptwerkzeug wird dem Spieler eine Möglichkeit zur Bearbeitung des Dungeons gegeben. So erlebt der Spieler die Möglichkeiten, die diese ungewöhnliche Art der vollständigen Umgebungsdarstellung bietet.

Es wäre nicht genug, dem Spieler einfach nur ein Werkzeug in die Hand zu geben und „Tob dich aus“ zu sagen. Das mag die Technologie zwar demonstrieren, aber hätte nichts mit einem Spiel zu tun. Deshalb gibt es den Schatzweiser. Er legt ein Ziel fest, zu dem der Spieler sich begeben soll.

Der Dichtescanner erfüllt gleich zwei verschiedene Aufgaben. Zum einen dient er als Orientierungswerkzeug ergänzend zum Schatzweiser. Wenn der Spieler sich verirrt und keinen Weg mehr findet, kann er die Materialbeschaffenheit in der Umgebung scannen und dadurch Orte finden, die sich nicht regenerieren, um dort eine Pause einlegen zu können, ohne vom Dungeon verschluckt zu werden oder um vorab Stellen aufzuspüren, an denen er nicht vorbeikommt, um sich frühzeitig drumherum einen Weg zu bahnen.

Zum anderen demonstriert der Dichtescanner aber noch einen technischen Aspekt des verwendeten Ray Marchings. Die Darstellung, die der Spieler normal sieht, ist nur eine Möglichkeit, wie das Render Ergebnis vom Ray Marching aussehen kann. Auf dem Display des Dichtescanners sieht er eine andere Variante, in der ein Teil komplett ausgeblendet wird und andere Teile hervorgehoben werden. Prinzipiell sind Berechnungen aus dem Ray Marching beliebig verarbeitbar. Der Dichtescanner ist ein Beispiel dafür, dass es unendlich viele Möglichkeiten gibt.

7.3 Ausrüstungsgegenstände

7.3.1 Die Lampe

Die Lampe wird permanent in der linken Hand geführt und unterliegt keinen Wechselmechaniken. Zur Steigerung der Immersion wird die Lichtintensität durch ein Skript mit zufälligen Werten leicht moduliert, was einen realistischen Flackereffekt simuliert. Eine technische Herausforderung stellt die Darstellung des Lampenglases dar. Standard-Transparenzshader in Unity kollidieren mit dem volumetrischen Ray-Marching-Verfahren. Entweder wird das Ray Marching hinter dem Glas vollständig ausgeblendet oder die Transparenz des Glases lässt sich nicht korrekt berechnen. Zur Lösung wird ein dedizierter Shader für das Glas implementiert, der eine, nach dem Ray Marching zwischengespeicherte Blittextur, also eine Textur des bisher gerenderten Bildes, bekommt. Dadurch lässt sich das Glas korrekt vor der Umgebung darstellen. Der Shader des Glases verzerrt zudem das erhaltene Bild anhand der Normal Map, um das Verhalten von Glas mit Kratzern realistisch darzustellen.



Abbildung 20: Lampe -Texturiertes 3D-Modell (Quelle: Eigene Darstellung)



Abbildung 21: Lampe - Zur Laufzeit (Quelle: Eigene Darstellung)

7.3.2 Granaten

Granaten dienen als primäres Werkzeug zur Wegbereitung. Nach dem Werfen folgt eine standardisierte Verzögerung von fünf Sekunden, bevor die Explosion automatisch ausgelöst wird. Durch Betätigung der rechten Maustaste kann der Spieler einen Fernzünder aktivieren, der alle aktuell aktiven Granaten sofort zur Detonation bringt. Die Explosion modifiziert die zugrundeliegende 3D-Textur, indem sie im Explosionsradius die Dichtewerte der betroffenen Voxel reduziert oder auf null setzt. Je nach dem im Materialkanal hinterlegten Widerstandswert bleibt die Struktur dauerhaft verändert oder beginnt, sich gemäß den im Compute Shader definierten Regenerationsparametern wieder aufzufüllen.



Abbildung 22: Granate - Texturiertes 3D-Modell (Quelle: Eigene Darstellung)



Abbildung 23: Granate - Zur Laufzeit (Quelle: Eigene Darstellung)

7.3.3 Schatzweiser

Der Schatzweiser orientiert sich funktional an einem Metalldetektor und dient der Orientierung im Dungeon. Die Ausrichtung des Geräts wird kontinuierlich mit der Richtung des Vektors zum Zielobjekt verglichen. Weist der Schatzweiser in Richtung des Schatzes, werden optische Signale (Blinken) und akustische Signale ausgelöst. Die Frequenz des visuellen Signals sowie die Lautstärke und Frequenz des akustischen Signals skalieren mit der Distanz zum Ziel. Dadurch erhält der Spieler eine kontinuierliche Rückmeldung über die Entfernung und Ausrichtung.



Abbildung 24: Schatzweiser - Texturiertes 3D-Modell (Quelle: Eigene Darstellung)

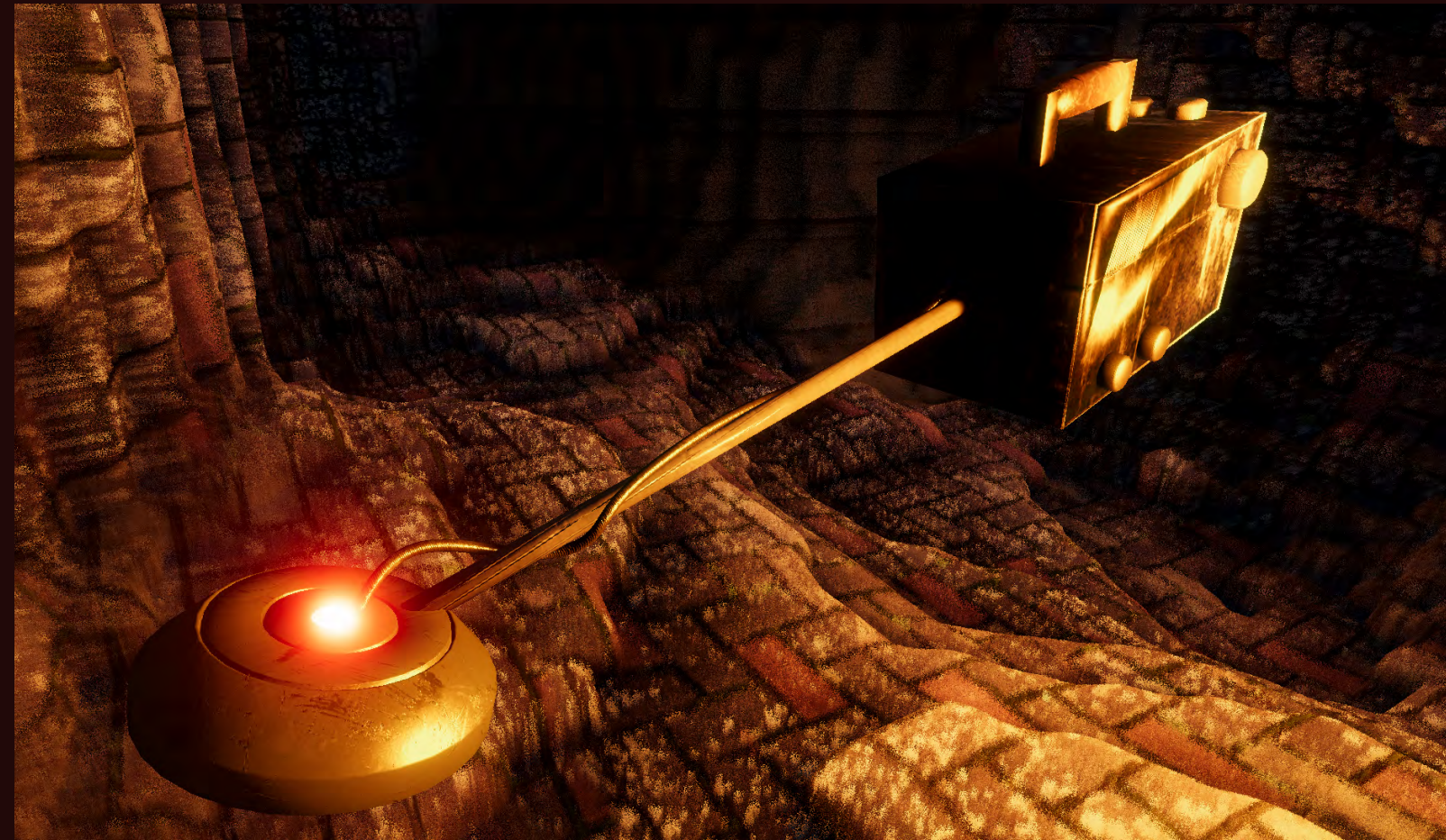


Abbildung 25: Schatzweiser - Zur Laufzeit (Quelle: Eigene Darstellung)

7.3.4 Dichtescanner

Der Dichtescanner visualisiert auf einem integrierten Display eine durchstrahlte Ansicht der Umgebung. Dabei wird das standardmäßig sichtbare, regenerierende Hauptmaterial des Dungeons gefiltert und ausgeblendet. Stattdessen werden ausschließlich Bereiche mit anderen Materialzuordnungen dargestellt, wodurch verborgene undurchdringbare Strukturen und sich nicht regenerierende Segmente erkennbar werden. Technisch wird diese Darstellung durch einen separaten Compute Shader realisiert. Dieser führt mit verringerter Auflösung einen zusätzlichen Ray Marching Durchlauf aus. Er filtert die Materialkanäle und schreibt das Ergebnis in eine Zwischentextur (eine Render Texture). Diese wird anschließend auf dem Display-Modell des Scanners angezeigt, wodurch eine performante, röntgenähnliche Visualisierung entsteht.

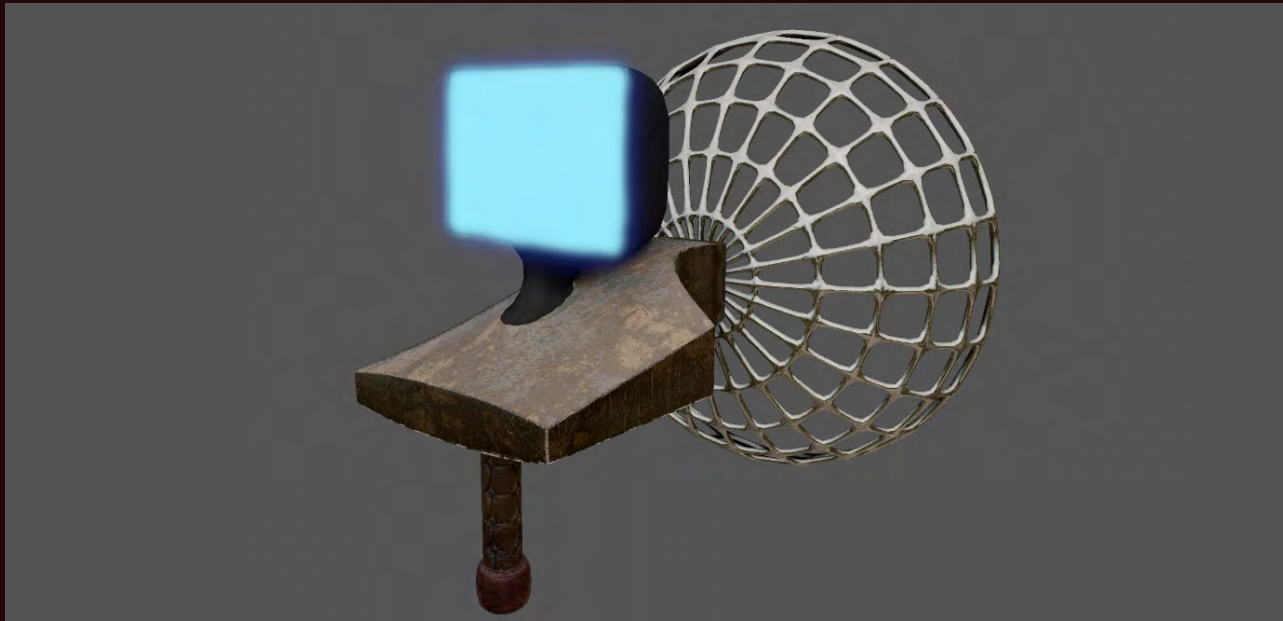


Abbildung 26: Dichtescanner - Texturiertes 3D-Modell (Quelle: Eigene Darstellung)

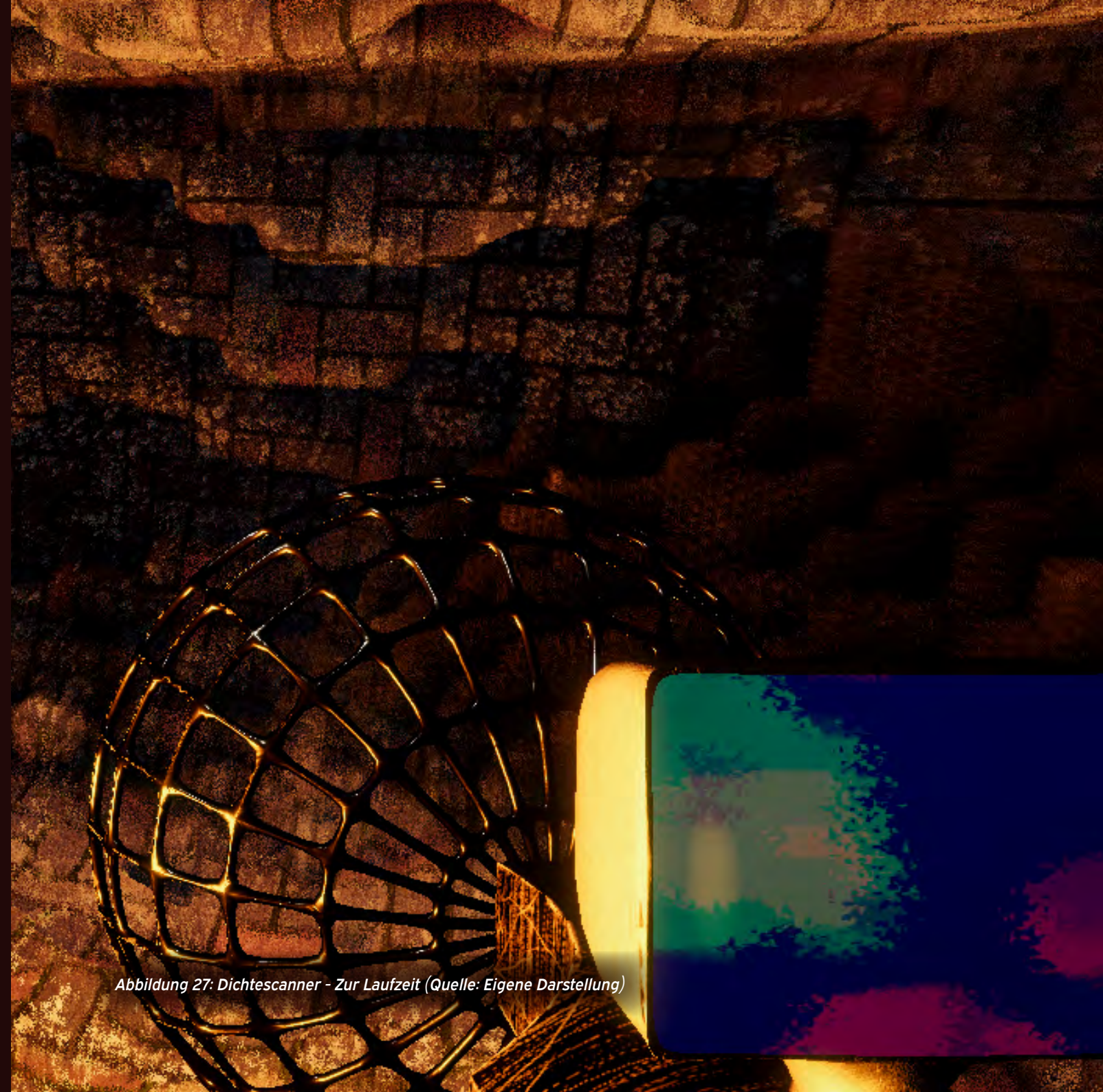


Abbildung 27: Dichtescanner - Zur Laufzeit (Quelle: Eigene Darstellung)

7.4 Zwischenfazit

Die GPU-basierte Volumenmodifikation durch Compute Shader und die kanalabhängige Materialsteuerung aus Kapitel 6 ermöglichen es, den Dungeon zur Laufzeit dynamisch zu verändern und diese Veränderungen direkt in das Gameplay zu übersetzen. Durch die Kombination von Granaten, einer Lampe sowie Schatzweiser und Dichtescanner entsteht ein exploratives Spielerlebnis, das auf der technischen Flexibilität des Ray Marching Ansatzes aufbaut. Die Implementierung zeigt, dass volumetrische Darstellungen nicht nur visuell, sondern auch mechanisch vollwertig in ein Spiel integriert werden können. Insbesondere der Dichtescanner demonstriert die vielseitige Einsetzbarkeit des Ray Marching Verfahrens für verschiedene Darstellungsweisen der Spielumgebung. Im abschließenden Kapitel werden die Ergebnisse evaluiert und die Grenzen sowie Perspektiven des Ansatzes diskutiert.

8 Ergebnisse

8.1 Auswertung

Visuell

Die Implementierung eines dynamischen Dungeon-Abschnitts mittels Ray Marching in Unity führt zu einer visuell kohärenten und technisch stabilen Darstellung. Die volumetrische Strahlverfolgung ermöglicht geschlossene, mehrschichtige Räume, die mit traditionellen Heightmap- oder Mesh-basierten Ansätzen nicht in dieser Flexibilität realisierbar wären. Die Kombination aus Triplanar Projection, approximierten Normalen und Ambient Occlusion erzeugt eine überzeugende räumliche Tiefenwirkung, ohne auf Normal Maps angewiesen zu sein. Die Integration von Licht und Schatten durch Directional und Point Lights, sowie die Verwendungsmöglichkeit von normalen 3D Meshes, schließen die Darstellung nahtlos in die Rendering Pipeline von Unity ein.

Technisch

Der Ansatz überzeugt technisch durch eine klare Arbeitsteilung zwischen CPU und GPU. Der Ray Marching Shader übernimmt die Strahlverfolgung effizient, während Compute Shader die zeitkritischen Aufgaben der Collider-Generierung, des Dungeon-Abbaus und der Regeneration verarbeiten. Die Nutzung von SphereCollidern in Kombination mit der FixedUpdate-Synchronisation gewährleistet eine stabile physikalische Interaktion mit echtzeitfähiger Latenz. Die Kanalabhängige Materialsteuerung erlaubt Variabilität in Widerstand und Regeneration des Dungeons,

was sich direkt im Gameplay niederschlägt. Insgesamt zeigt diese Arbeit, dass Ray Marching unter Einhaltung strenger Optimierungen eine leistungsfähige Alternative zu statischen Geometrien darstellt.

Die Echtzeitlauffähigkeit ist auf sehr moderner Hardware uneingeschränkt gegeben. Auf einem Testsystem mit leistungsfähigen Komponenten erreichte die Build-Variante der Demo durchgängig mehr als 60 Bilder die Sekunde.

Bei einem Test auf einem System mit teilweise älterer Hardware zeigte sich aber eine Schwäche: Die Grafikkarte muss für diese Demo sehr leistungsfähig sein. Eine ältere Grafikkarte, in dem Fall eine NVIDIA GTX 1080 aus dem Jahr 2016, brachte nur eine Bildrate von ungefähr 25 Bildern die Sekunde zustande und hatte in bestimmten Situationen sogar Geschwindigkeitsabfälle auf unter 10 Bilder pro Sekunde. Und das sogar bei einer niedrigeren getesteten Auflösung von 1920 x 1080 Pixeln im Vergleich zu 2560 x 1440 auf dem modernen System.

Da die Demo auf aktueller Hardware die Anforderungen aber erfüllt, gilt das Performanceziel als erreicht.

Gameplay

Das gewählte Gameplay-Konzept für die Demo und die Umsetzung sind eine gelungene Unterstützung zur Demonstration der Technik. Der Spieler kann mit den Ausrüstungsgegenständen

die Umgebung verändern, ein Ziel finden und die Umgebung auf ihre Materialbeschaffenheit durch Strukturen hindurch untersuchen. Das gewählte Konzept ist simpel genug, sodass es in der begrenzten Zeit umsetzbar war, fügt aber dem Ergebnis noch eine Ebene hinzu, die dem Spieler einen Vorgeschmack bietet, was für Möglichkeiten die Technik eröffnet.

Dennoch handelt es sich bei der Demo um kein vollwertiges Spiel. Mit den Ausrüstungsgegenständen ist zwar eine Grundlage geschaffen, um darauf ein Spiel aufzubauen, aber es fehlt an Ausarbeitung für längeren Spielspaß, der über einen wow-Effekt über die Verwendung einer ungewöhnlichen Technik hinausgeht.

8.2 Grenzen des Ansatzes

Trotz der technischen Umsetzbarkeit weist der Ansatz spezifische Limitierungen auf, die bei einer Weiterentwicklung berücksichtigt werden müssen.

Detailgrenzen

Die Darstellungsgüte hängt stark von der Auflösung der zugrundeliegenden 3D-Texturen ab. Höhere Auflösungen erhöhen die visuelle Qualität oder würden größere Dungeon-Abschnitte bei gleichbleibender Qualität zulassen, aber sind sowohl für die Speicherbelegung als auch für die Verarbeitungsgeschwindigkeit nicht trivial. Zur Verdeutlichung: In diesem Projekt gibt es zwei

3D-Texturen. Jede davon ist 256 x 256 x 256 Pixel groß. Wenn man die Abmessungen verdoppeln würde, würde sich die Größe der 3D-Textur verachtfachen. Und das bei jeder Verdopplung. Bei normalen 2D Texturen sind Maße wie 2K (2048 x 2048) oder 4K (4096 x 4096) übliche Werte. Eine 3D-Textur mit den Maßen 4096 x 4096 x 4096 wäre aber 4096 mal so groß wie eine einzige 4K Textur. Die Größe einer unkomprimierten 3D-Textur skaliert also kubisch mit der Anzahl Pixel pro Kante (vgl. R. Arbore, 2024, p. 126).

Eine weitere Stellschraube ist die Auflösung des Ausgabebildes. Aktuell wird der Fullscreen Ray Marching Shader für jeden Pixel des Bildes ausgeführt. Für den Fall, dass jemand einen hochauflösenden Monitor zusammen mit einer zu schwachen Grafikkarte verwendet, würde der Rechenaufwand steigen und die Flüssigkeit der Darstellung nicht mehr gewährleistet sein. Es wäre aber möglich (aber in dieser Demo nicht implementiert), das Ray Marching mit einer geringeren Auflösung als der Bildschirmauflösung zu rendern oder gleich das ganze Spiel auf einer niedrigeren Auflösung zu rendern und eine der modernen Upscaling Technologien wie NVIDIA DLSS (vgl. NVIDIA Corporation, 2026) oder AMD FSR (vgl. Advanced Micro Devices, Inc., 2026) zu implementieren.

Physikungenaugigkeit

Die Kollisionsbehandlung basiert auf einer radiusbasierten SphereCollider Generierung. Dies gewährleistet Performance, schränkt jedoch die Präzision ein. Objekte mit komplexen Collidern können an Stellen des Dungeons kollidieren, an denen visuell eigentlich noch keine Kollision sein dürfte. Eine vollständige, meshbasierte Collider-Generierung wäre präziser, würde jedoch erheblich mehr Rechenleistung beanspruchen und die Echtzeitfähigkeit gefährden.

Beleuchtungseinschränkung

Zudem bleibt der Zugriff auf Additional Lights im Fullscreen-Shader auf das nächstgelegene Licht beschränkt. Eine vollständige Beleuchtung durch mehrere dynamische Lichtquellen erfordert entweder eine manuelle Datenübergabe oder eine Anpassung der Rendering Pipeline.

Materialbeschaffenheit

In der Demonstration werden bewusst nur matte Oberflächen verwendet, denn in dem hier vorgestellten Ansatz sind glänzende oder spiegelnde Oberflächen nicht vorgesehen. Die approximierten Normalen sind dafür schlicht zu ungenau. Mit genaueren Normalen, wie sie bei SDFs vorhanden sein oder berechnet werden können, ließe sich das jedoch bewerkstelligen. Beim Verzicht auf die Voxel würde man aber die Flexibilität der freien Umgebungsänderbarkeit einbüßen.

8.3 Zusammenfassung

Diese Arbeit hat gezeigt, dass Ray Marching als Grundlage für dynamische, interaktive Spielumgebungen in Unity technisch machbar und visuell überzeugend ist. Durch die Kombination eines Fullscreen-Shader-Ansatzes, GPU-basierter Compute Shader für Kollisionen und Volumenmodifikationen sowie kanalbasierter Materialsteuerung lässt sich ein veränderlicher Dungeon-Abschnitt realisieren, der traditionelle Mesh- und Terrain-Systeme in Flexibilität und Laufzeitveränderlichkeit übertrifft. Die Implementierung demonstriert, wie volumetrisches Rendering, physikalische Interaktion und gameplayrelevante Instrumente in einer kohärenten Architektur zusammengeführt werden können. Die erzielten Ergebnisse bestätigen, dass Ray Marching über atmosphärische Effekte hinaus ein funktionsfähiges Fundament für räumliche Spielstrukturen bietet.

Jedoch ist die Darstellung von Umgebungen mit Ray Marching keine allmächtige Technik, die zukünftig jegliches standardisierte Rendering ersetzen wird. Für die Echtzeitlauffähigkeit ist gute, moderne Hardware nötig und die Genauigkeit in der visuellen Darstellung, der Physik und der Darstellbarkeit von glänzenden Materialien ist mit dem in dieser Arbeit behandelten Verfahren eingeschränkt.

8.4 Ausblick

8.4.1 Weiterentwicklung zum vollwertigen Spiel

Die Weiterentwicklung zum vollwertigen Spiel erfordert vorrangig die Erweiterung des Gameplay-Designs, die technische Grundlage ist mit dieser Arbeit geschaffen. Für die Erweiterung des Gameplays bieten sich die Integration von weiteren sammelbaren Gegenständen, die dem Spieler verschiedene Vorteile bieten, Gegner, die in dem Dungeon auftauchen und bekämpft werden müssen, ein Fortschrittssystem, durch das man sich mit anderen Spielern vergleichen kann (beispielsweise: Wer hat die meisten/wertvollsten Schätze im Dungeon gesammelt), sowie das Hinzufügen weiterer Dungeon-Abschnitte als Level an. Auch wäre es möglich, mehrere Dungeon-Abschnitte aneinander zu platzieren, die nur gerendert werden, wenn sie nicht verdeckt sind, wodurch praktisch unendlich große Dungeons aus vielen aneinander platzierten Dungeon-Abschnitten möglich wären.

Da es sich hierbei nur um eine Demo handelt, wären neben Erweiterungen grundsätzlich auch noch einige Feinschliffe nötig, die es aus Zeitgründen nicht in die Abgabe geschafft haben. Dazu zählen sowohl visuelle Ausarbeitungen als auch das Sound Design. Sound Effekte wie Umgebungs-, Bewegungs-, Ausrüstungs-, Explosions- und Kollisionsgeräusche würden der Demo ebenfalls in Richtung eines vollwertigen Spiels verhelfen.

8.4.2 Technische Tools

Um den Dungeon-Abschnitt zu erstellen ist als „Nebenprodukt“ die Möglichkeit entstanden, 3D-Texturen zu bearbeiten. Es wäre möglich, diese Funktion als eigenes Programm zu separieren, um ein 3D Tool zu schaffen, mit dem man 3D-Texturen direkt visuell gestalten kann. Auch eine Bearbeitung der einzelnen Kanäle der 3D-Textur ist dadurch möglich.

Die 3D-Textur des Dungeon-Abschnitts im mitgelieferten Build ist zwar durch das Verfahren aus 3.5 entstanden, aber die 3D-Textur für die Materialzuordnung ist bereits durch eigene Funktionen direkt in Unity bearbeitet.

Literaturverzeichnis

Advanced Micro Devices, Inc., 2026. Unity- AMD GPUOpen. [Online] Available at: <https://gpuopen.com/unity/> [Zugriff am 10 08 2026].

Brian Wyvill, G. W., 1989. Field functions for implicit surfaces. *The Visual Computer*, pp. 75 - 82.

Chris Green, V., 2007. Improved Alpha-Tested Magnification for Vector Textures and Special Effects. *Advanced Real-Time Rendering in 3D Graphics and Games Course - SIGGRAPH 2007*, pp. 9 - 18.

Epic Games, 2026. Volumetric Cloud Component in Unreal Engine | Unreal Engine 5.8. [Online] Available at: <https://dev.epicgames.com/documentation/unreal-engine/volumetric-cloud-component-in-unreal-engine> [Zugriff am 16 08 2026].

Hart, J. C., 1995. Sphere Tracing: A Geometric Method for the Antialiased Ray Tracing of Implicit Surfaces. *The Visual Computer*, 06, pp. 70 - 119.

Hello Games, 2024. Worlds Part I Update - No Man's Sky. [Online] Available at: <https://www.nomanssky.com/worlds-part-i-update/> [Zugriff am 22 08 2026].

Lague, S., 2019. Coding Adventure: Clouds. [Online] Available at: <https://www.youtube.com/watch?v=4QOcCGI6xOU> [Zugriff am 21 08 2026].

Larsson, J., kein Datum Raymarching The Gunk. [Online] Available at: <https://jarllarsson.github.io/gen/gunkraymarcher.html> [Zugriff am 22 08 2026].

NVIDIA Corporation, 2026. NVIDIA DLSS | NVIDIA Developer. [Online] Available at: <https://developer.nvidia.com/rtx/dlss> [Zugriff am 10 08 2026].

R. Arbore, J. L. A. W. S. G. E. S., 2024. Hybrid Voxel Formats for Efficient Ray Tracing. In: *Advances in Visual Computing*. Lake Tahoe, NV, USA: Springer, pp. 125 - 138.

S. M. Keerthana, M. R. M. V., 2025. 3D Model Construction from 2D CT Slices Using Window Centering, Marching Cubes, and Ray Casting for Medical Imaging. *IEEE 4th International Conference for Advancement in Technology (ICONAT)*, 19-21 09, pp. 1-8.

Samuli Laine, T. K., 2011. High-Performance Software Rasterization on GPUs. *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics, Band HPG '11 High Performance Graphics*, pp. 79 - 88.

Simon Green, N., 2005. *Volume Rendering For Games*. San Francisco, Kalifornien, Game Developers Conference.

Smith, A., 2025. Unity 6.3 LTS is Now Available. [Online] Available at: <https://unity.com/blog/unity-6-3-lts-is-now-available> [Zugriff am 04 06 2026].

Unity Technologies, 2023. About SDF fonts | TextMeshPro | 4.0.0-pre.2. [Online] Available at: <https://docs.unity3d.com/Packages/com.unity.textmeshpro@4.0/manual/FontAssetsSDF.html> [Zugriff am 16 08 2026].

Unity Technologies, 2023a. Triplanar Node | Shader Graph | 6.9.2. [Online] Available at: <https://docs.unity3d.com/Packages/com.unity.shadergraph@6.9/manual/Triplanar-Node.html> [Zugriff am 19 07 2026].

Unity Technologies, 2023b. Volumetric Clouds | High Definition RP | 12.0.0. [Online] Available at: <https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@12.0/manual/Override-Volumetric-Clouds.html> [Zugriff am 16 08 2026].

Unity Technologies, 2026a. Unity - Manual: 3D textures. [Online] Available at: <https://docs.unity3d.com/6000.5/Documentation/Manual/class-Texture3D.html> [Zugriff am 11 08 2026].

Unity Technologies, 2026b. Unity - Manual: Collider types and performance. [Online] Available at: <https://docs.unity3d.com/6000.0/Documentation/Manual/physics-optimization-cpu-collider-types.html> [Zugriff am 19 07 2026].

Unity Technologies, 2026c. Unity - Manual: Event function execution order. [Online] Available at: <https://docs.unity3d.com/6000.1/Documentation/Manual/execution-order.html> [Zugriff am 19 07 2026].

Unity Technologies, 2026d. Unity - Manual: Fixed updates. [Online] Available at: <https://docs.unity3d.com/6000.1/Documentation/Manual/fixed-updates.html> [Zugriff am 19 07 2026].

Unity Technologies, 2026e. Unity - Manual: Render additional lights in a shader in URP. [Online] Available at: <https://docs.unity3d.com/6000.1/Documentation/Manual/urp/use-built-in-shader-methods-additional-lights-fplus.html> [Zugriff am 19 07 2026].

Unity Technologies, 2026f. Unity - Manual: Working with Heightmaps. [Online] Available at: <https://docs.unity3d.com/6000.4/Documentation/Manual/terrain-Heightmaps.html>

Quellenzugang

Die Quellen aus dem Literaturverzeichnis sind direkt frei zugänglich oder können über eine Verbindung mit dem VPN der Hochschule im Internet abgerufen werden.

Übersicht verwendeter Hilfsmittel

Hilfsmittel	Genutzt für
ChatGPT	Einstieg ins Thema, Verständnisfragen, Programmierhilfe
Qwen 3.5 (Lokal)	Gliederungshilfe, Formulierungshilfe, Literatur nach relevanten Stellen durchsuchen
Qwen 3.6 (Lokal)	Unterstützung bei Gliederung und Formulierung der Arbeit

Übersicht verwendeter Programme

Programm	Genutzt für
Microsoft Word	Verfassen der Bachelorarbeit
Unity 6	Basis für die praktische Arbeit
Blender	Modellierung von 3D Objekten
Armorpaint	Texturierung von 3D Objekten
GIMP	Bildbearbeitung

